UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS

FACULTAD DE COMERCIO, ADMINISTRACIÓN Y CIENCIAS SOCIALES

TEMAS SELECTOS DE TRATAMIENTOS DE INFORMACIÓN

Proyecto unidad 2

Athala Dulce Edith Gómez Arellano

Monica Fernanda Alfaro Rangel

Diana Maritza Mendoza Costilla

9 F

NVO. LAREDO, TAMPS FECHA DE ENTREGA

12/11/2026

Índice

Introducción 4

Proyecto unidad 1 5

Objetivo 5

Contenido: 5

1.Descripción de la empresa real 5

Giro 5

Ubicación 5

Misión 5

Visión 5

Productos o servicios que ofrece 6

Características generales de sus operaciones 6

2. Descripción del problema de la empresa relacionada con la administración de sus datos 6

3. Necesidades de información de la empresa 7

Tipo de procesamiento requerido 7

Nivel de disponibilidad necesario 7

Necesidades de almacenamiento 8

Crecimiento esperado 8

Necesidades de seguridad 8

Tipo de escalabilidad requerida 9

Principales riesgos relacionados con la información 9

Cantidad de usuarios y/o departamentos 10

4. Desarrollar la problemática utilizando el modelo CIA en un gráfico 10

5. Infraestructura del servidor de base de datos: 10

Configuración propuesta: 11

Ventajas y desventajas 12

6. Diseño de la base de datos, grupo de archivos rheemSecundario, su archivo .ndf y mínimo 8 tablas (código T-SQL) 13

7. Definir al menos tres tipos de usuarios o puestos que utilizarán el sistema: 15

8. Crear una matriz de privilegios 16

9. Emplear el principio del mínimo privilegio: 16

10.Diseñar una estrategia de respaldo: 19

11.Definan el RPO de la empresa. 23

12.Definan el RTO de la empresa 23

13.Realicen una prueba de respaldo y restauración: 24

14.Diseñen un plan de mantenimiento. 26

15. Definan qué información deberá ser auditada. 27

Acciones principales que serán auditadas 28

16. Implementen mecanismos de auditoría. 29

Eventos que se registrarán 29

Proyecto unidad 2 33

1. Problemática y diseño de datos 33

1.1 Problemática de la empresa 33

1.2 Diseño de los datos 33

1.3 Tablas seleccionadas para el análisis 33

1.4 Reglas de calidad de los datos 34

1.5 Importancia de mejorar la calidad de los datos 35

2. Creación de los archivos csv 35

2.1. Código Python 36

2.2 Diagnóstico de la calidad de los datos. 39

2.3 Establece las reglas de negocio concretas en cada columna por cada archivo (puede ser una tabla): rangos aceptables, obligatoriedad, consistencia entre campos y relaciones entre archivos. 41

3. Reglas de negocio de proveedores.csv 42

2.4 Limpieza y transformación de datos 42

2.9 Ejecuta 3 consultas aplicando filtros (where), ordenamiento y columna calculada (SUM, AVG, MIN, MAX, etc.) 58

Conclusión 63

Referencias 64

Introducción

En este proyecto vamos a continuar trabajando con la base de datos de la empresa Rheem, pero ahora también vamos a utilizar Python para revisar que la información esté correcta y bien organizada. Para esto, trabajaremos con archivos csv de productos, inventarios y proveedores, en los que habrá algunos errores, como datos repetidos, campos vacíos, precios incorrectos y teléfonos o correos mal escritos.

Con ayuda de Python, vamos a identificar estos errores, revisar qué porcentaje de registros están bien y cuáles necesitan corregirse. También realizaremos una limpieza de los datos para mejorar la información y guardarla en nuevos archivos.

La idea es aprender cómo Python puede ayudarnos a detectar y corregir problemas en los datos antes de pasarlos a SQL Server. Así, la empresa podrá tener información más confiable, facilitar sus actividades y evitar problemas al momento de consultar sus productos, controlar sus inventarios o trabajar con sus proveedores.

Proyecto unidad 1

Objetivo

Diseñar e implementar una propuesta de administración de base de datos para una empresa real, partiendo de una problemática hipotética creada por el equipo, y aplicando los conocimientos adquiridos durante la Unidad I para justificar decisiones relacionadas con infraestructura, almacenamiento, seguridad, respaldos, recuperación, mantenimiento y auditoría.

Contenido:

1.Descripción de la empresa real

Nombre de la empresa. RHEEM

Giro

RHEEM pertenece al sector de fabricación y comercialización de productos relacionados con la climatización, calefacción y suministro de agua caliente.

Ubicación

Para el proyecto se tomará como referencia la sucursal ubicada en:

C. Los Dos Laredos 16, Río Bravo, 88185 Nuevo Laredo, Tamps.

Misión

Para el desarrollo de este proyecto, se considera que la misión de RHEEM consiste en ofrecer soluciones de climatización, calefacción y calentamiento de agua que respondan a las necesidades de hogares y negocios, procurando confiabilidad, eficiencia y buen desempeño. Esta descripción se basa en la información institucional de la empresa (Rheem Manufacturing Company, s. f.).

Visión

La visión de RHEEM se puede enfocar en mantenerse como una empresa líder en sistemas HVAC y calentamiento de agua, utilizando innovación, eficiencia, sostenibilidad y buscando la satisfacción de sus clientes. (Company, s.f.)

Productos o servicios que ofrece

Entre los principales productos que maneja RHEEM se encuentran:

- Calentadores de agua.
- Sistemas de calefacción.
- Equipos de aire acondicionado.
- Sistemas de ventilación.
- Equipos para el confort del hogar.
- Equipos para aplicaciones comerciales.

Características generales de sus operaciones

RHEEM realiza actividades relacionadas con la fabricación, comercialización y venta de equipos para climatización y calentamiento de agua. Para llevar a cabo estas actividades necesita manejar información de productos, ventas, empleados, proveedores e inventario, así como los datos de los compradores asociados a las operaciones de venta.

Por lo tanto, es importante que la información se encuentre organizada y disponible para los diferentes departamentos de la empresa.

2. Descripción del problema de la empresa relacionada con la administración de sus datos

La problemática que se propone para RHEEM consiste en que puede existir una mala organización de la información dentro de su base de datos, principalmente por la duplicidad e inconsistencia de algunos registros.
Un ejemplo es que los nombres de los productos pueden registrarse de diferentes maneras, aunque se trate del mismo artículo. Esto puede provocar dificultades para identificar los productos y relacionarlos correctamente con sus registros de inventario.
También pueden presentarse errores en los datos de las compras, las ventas y las existencias, lo que dificulta mantener la información actualizada y generar reportes confiables.

Estos problemas pueden ocasionar:

- Información duplicada o inconsistente.
- Errores en los registros de productos e inventarios.

- Dificultad para encontrar información.
- Problemas para generar reportes.
- Dificultad para actualizar los datos.
- Diferencias entre las existencias registradas y las reales.
- Posibles errores al momento de tomar decisiones.

3. Necesidades de información de la empresa

Tipo de procesamiento requerido

Para RHEEM se considera necesario utilizar OLTP y OLAP.

OLTP: se utilizaría para las operaciones diarias de la empresa, por ejemplo:

- Registrar ventas.
- Registrar productos.
- Registrar ventas.
- Actualizar el inventario.
- Registrar proveedores.
- Actualizar la información de las operaciones comerciales.

OLAP: se utilizaría para analizar la información y apoyar la toma de decisiones, por ejemplo:

- Analizar las ventas por mes.
- Identificar los productos más vendidos.
- Analizar las ventas por sucursal.
- Conocer cuáles productos tienen mayor demanda.
- Comparar las ventas de diferentes periodos.

Nivel de disponibilidad necesario

RHEEM necesita un nivel de disponibilidad alto, principalmente durante el horario de operación.

La información de clientes, productos, ventas e inventarios debe estar disponible para que los empleados puedan realizar sus actividades sin interrupciones.

Se propone contar con:

- Respaldos periódicos.
- Recuperación ante fallas.
- Monitoreo de la base de datos.
- Mantenimiento preventivo.
- Protección contra pérdida de información.

Necesidades de almacenamiento

La base de datos deberá almacenar información relacionada con:

- Productos.
- Ventas.
- Detalles de ventas.
- Inventarios.
- Proveedores.
- Empleados.
- Sucursales.
- Direcciones.

Crecimiento esperado

Se espera un crecimiento constante de la base de datos debido al registro de nuevos clientes, productos, ventas y movimientos de inventario.

Por ejemplo, cada día se pueden generar nuevos registros de ventas y clientes. A largo plazo, esta información puede representar una cantidad considerable de datos.

Por esta razón, se recomienda dejar capacidad disponible y utilizar una estrategia de crecimiento que permita ampliar el almacenamiento sin afectar las operaciones.

Necesidades de seguridad

La información de RHEEM debe protegerse para evitar accesos no autorizados, modificaciones incorrectas o pérdida de información.

- Se propone implementar:
- Usuarios con diferentes permisos.
- Contraseñas seguras.
- Control de acceso por departamentos.
- Respaldos periódicos.
- Permisos de lectura y escritura según las funciones de cada empleado.
- Auditoría de cambios importantes.
- Protección de información de clientes y ventas.

Tipo de escalabilidad requerida

Se recomienda principalmente una escalabilidad vertical y posteriormente horizontal.

La escalabilidad vertical permitiría aumentar los recursos del servidor, como:

- Memoria RAM.
- Procesador.
- Almacenamiento.

Principales riesgos relacionados con la información

- Los principales riesgos identificados para RHEEM serían:
- Duplicidad de información, debido a registros repetidos.
- Pérdida de información por fallas del sistema o hardware.
- Acceso no autorizado a información de clientes y ventas.
- Modificación incorrecta de datos por parte de usuarios.
- Fallas del servidor.
- Ataques informáticos o malware.
- Falta de respaldos actualizados.
- Errores humanos al capturar o modificar información.
- Crecimiento descontrolado de la base de datos.
- Inconsistencia de información entre diferentes registros.

Cantidad de usuarios y/o departamentos

Departamento	Usuarios aproximados	Información que utilizan
Ventas	8	Clientes, productos y ventas
Almacén	6	Inventario y productos
Administración	4	Ventas y reportes
Compras	4	Proveedores y productos
Servicio técnico	4	Clientes y equipos
Gerencia	2	Reportes y análisis
Sistemas / BD	2	Administración y seguridad
Total	**30**	

Tabla 1. Cantidad de usuarios y/o departamentos

4. Desarrollar la problemática utilizando el modelo CIA en un gráfico

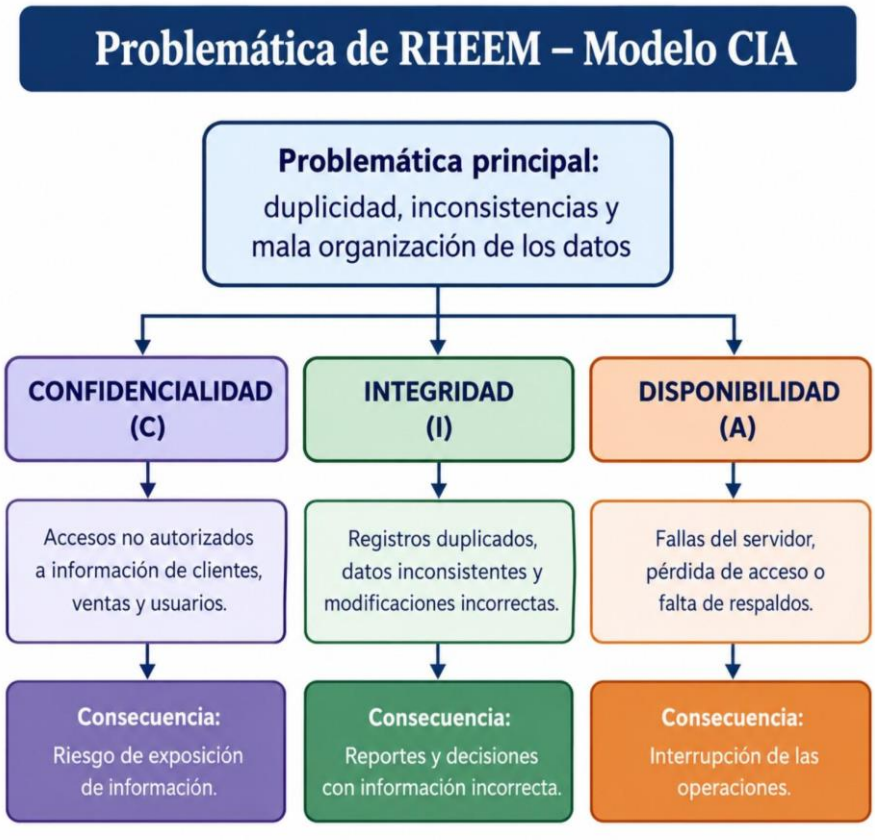


Figura 1. Mapa conceptual.

5. Infraestructura del servidor de base de datos:

Se propone utilizar un servidor dedicado virtual (VPS) o una máquina virtual administrada para alojar SQL Server en un entorno controlado. Para la práctica escolar puede mantenerse una

instancia local; sin embargo, para un escenario empresarial el servidor debe contar con acceso restringido, monitoreo, respaldo externo y un procedimiento de recuperación probado. La elección definitiva dependerá del presupuesto, la cantidad real de transacciones y la edición de SQL Server disponible.

Configuración propuesta:

Elemento	Propuesta inicial	Justificación
Tipo de servidor	Servidor dedicado virtual o máquina virtual administrada	Permite centralizar la base de datos y controlar accesos y recursos.
Edición de SQL Server	SQL Server Standard para un entorno empresarial; Developer solo para desarrollo/pruebas	La edición debe elegirse según licenciamiento y funciones requeridas. Developer no debe usarse como servidor de producción.
CPU	4 vCPU como punto de partida	Suficiente como estimación inicial para una carga pequeña o mediana; debe ajustarse con mediciones.
Memoria RAM	16 GB como punto de partida	Permite reservar memoria para el motor y el sistema operativo; se debe monitorear el uso real.
Almacenamiento	SSD; separar, si es posible, datos, log y respaldos	Mejora el rendimiento y facilita administrar espacio y recuperación.
RAID	RAID 1 para dos discos de datos o RAID 10 si se requieren más discos y rendimiento	RAID 1 tolera la falla de un disco del espejo; RAID 10 combina espejo y distribución, pero necesita al menos cuatro discos.

Número de discos	2 para RAID 1; 4 o más para RAID 10	La cantidad depende de la configuración elegida y del crecimiento esperado.
Respaldos	Destino independiente del servidor y una copia externa	RAID no sustituye los respaldos: protege frente a ciertas fallas de disco, no frente a borrados, corrupción o incidentes que afecten todas las copias.

Tabla 2. Propuesta.

Ventajas y desventajas

Ventajas: administración centralizada, control de permisos, monitoreo, posibilidad de ampliar CPU/RAM y una política de respaldos programada.
Desventajas: costo de infraestructura y licencias, necesidad de personal capacitado y dependencia de conectividad si el servidor se aloja fuera de las instalaciones.

Si falla un disco en un arreglo RAID 1, el sistema puede continuar operando desde el disco que permanece funcional, siempre que el arreglo y el controlador trabajen correctamente; se debe reemplazar el disco dañado y reconstruir el espejo cuanto antes. RAID 10 puede tolerar ciertas fallas, dependiendo de qué discos fallen. Si falla más de un disco de forma incompatible con el arreglo, puede perderse el servicio o la información, por lo que siguen siendo indispensables los respaldos.

Esta propuesta es adecuada para la problemática planteada porque centraliza la información de productos, inventarios, proveedores, empleados y operaciones; facilita el control de accesos y permite planear respaldo, restauración, mantenimiento y auditoría. Los valores de CPU, RAM y discos son una estimación académica y deben validarse con pruebas de carga.

6. Diseño de la base de datos, grupo de archivos rheemSecundario, su archivo .ndf y mínimo 8 tablas (código T-SQL)

```
---CREACION DE BASE DE DATOS

CREATE DATABASE rheem
ON PRIMARY
(
NAME = rheemData,
FILENAME = 'C:\data\rheemData.mdf',
SIZE= 50MB,
FILEGROWTH= 25%)
LOG ON
(
NAME = rheemLog,
FILENAME = 'C:\data\rheemLog.ldf',
SIZE = 20MB,
FILEGROWTH = 10MB
);

--AGREGAR GRUPO DE ARCHIVO SECUNDARIO
ALTER DATABASE rheem
ADD FILEGROUP rheemSecundario;
```

Figura 2. Crear una base de datos.

```
--ARCHIVO .NDF
ALTER DATABASE rheem
ADD FILE
(
NAME = rheemSecData,
FILENAME = 'C:\data\rheemSecData.ndf',
SIZE = 30MB,
FILEGROWTH = 10MB
)
TO FILEGROUP rheemSecundario;
SELECT name
FROM sys.filegroups;
--TABLAS
USE rheem;
CREATE TABLE departamentos (
id_departamentos INT PRIMARY KEY IDENTITY (1,1),
nombre VARCHAR(100) NOT NULL,
descripcion VARCHAR (100) NOT NULL,
)ON [PRIMARY];
```

Figura 3. Archivo .ndf.

```sql
--EMPLEADOS
CREATE TABLE empleados (
id_empleado INT PRIMARY KEY IDENTITY (1,1),
nombre VARCHAR (100) NOT NULL,
apellido VARCHAR (100) NOT NULL,
puesto VARCHAR (100) NOT NULL,
id_departamento INT, CONSTRAINT FK_empleados_departamentos
FOREIGN KEY (id_departamento)
REFERENCES departamentos(id_departamentos)
)ON [PRIMARY];

--USUARIOS
CREATE TABLE usuarios
(
id_suario INT PRIMARY KEY  IDENTITY(1,1) ,
nombre_usuario VARCHAR(50) NOT NULL UNIQUE,
id_empleado INT,
activo BIT NOT NULL DEFAULT 1,
CONSTRAINT FK_usuarios_empleados
FOREIGN KEY (id_empleado)
REFERENCES empleados(id_empleado)
)ON [PRIMARY];
```

Figura 4. Tablas.

```sql
--proveedores
CREATE TABLE proveedores
(
id_proveedor INT PRIMARY KEY  IDENTITY(1,1),
nombre VARCHAR(150) NOT NULL,
telefono VARCHAR(20)NOT NULL,
correo VARCHAR(100)NOT NULL
)ON [PRIMARY];

--PRODCUTOS
CREATE TABLE productos
(
id_producto INT PRIMARY KEY  IDENTITY(1,1),
nombre VARCHAR(150) NOT NULL,
categoria VARCHAR(100),
precio DECIMAL(12,2) NOT NULL,
activo BIT NOT NULL DEFAULT 1
)ON [PRIMARY];
```

Figura 5. Tablas.

```
--INVENTARIOS
CREATE TABLE inventarios
(
id_inventario INT PRIMARY KEY  IDENTITY(1,1),
id_producto INT NOT NULL,
existencia INT NOT NULL DEFAULT 0,
ubicacion VARCHAR(100),
CONSTRAINT FK_Inventarios_Productos FOREIGN KEY (id_producto)
REFERENCES Productos(id_producto)
) ON rheemSecundario;

--7.COMPRAS
CREATE TABLE Compras
(
id_compra INT PRIMARY KEY  IDENTITY(1,1),
id_proveedor INT NOT NULL,
fecha DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
total DECIMAL(12,2) NOT NULL,
CONSTRAINT FK_Compras_Proveedores
FOREIGN KEY (id_proveedor)
REFERENCES Proveedores(id_proveedor)
) ON rheemSecundario;
```

Figura 6. Tablas.

```
-- TABLA 8: Ventas
-- Grupo de archivos: RHEEM_SECUNDARIO
CREATE TABLE Ventas
(
id_venta INT PRIMARY KEY  IDENTITY(1,1),
fecha DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
cliente VARCHAR(150),
total DECIMAL(12,2) NOT NULL
) ON rheemSecundario;
```

Figura 7. Tablas.

7. Definir al menos tres tipos de usuarios o puestos que utilizarán el sistema:

1. Encargado de almacén

- Consulta los productos y sus existencias.
- Registra entradas y salidas de inventario.
- No administra usuarios ni permisos.

2. Encargado de compras y ventas

- Registra compras y ventas.
- Consulta productos y proveedores.
- No modifica los permisos de otros usuarios.

3. Administrador de base de datos

- Administra usuarios y roles.
- Realiza respaldos y restauraciones.
- Supervisa el mantenimiento y la auditoría.

8. Crear una matriz de privilegios

Tablas	Almacén	Compras/Ventas	DBA
Departamentos	SELECT	—	Administración total
Empleados	—	—	Administración total
Usuarios	—	—	Administración total
Proveedores	SELECT	SELECT, INSERT, UPDATE	Administración total
Productos	SELECT	SELECT	Administración total
Inventarios	SELECT, INSERT, UPDATE	SELECT	Administración total
Compras	SELECT	SELECT, INSERT	Administración total
Ventas	SELECT	SELECT, INSERT	Administración total

Tabla 3. Crear una matriz de privilegios.

9. Emplear el principio del mínimo privilegio:

Creación del login

```
--Crear los logins
CREATE LOGIN almacen
WITH PASSWORD = 'Almacen_Rheem_123!';

CREATE LOGIN compras
WITH PASSWORD = 'Compras_Rheem_456!';

CREATE LOGIN dba
WITH PASSWORD = 'Dba_Rheem_789!';
```

Figura 8. Logins.

Creación del usuario

```
--Crear usuarios
CREATE USER almacen
FOR LOGIN almacen;

CREATE USER compras
FOR LOGIN compras;

CREATE USER dba
FOR LOGIN dba;
```

Figura 9. Usuarios.

Asignación de roles y/o de los privilegios (si será de forma personalizada crear rol)

```
-- Encargado de almacén: consultar información
ALTER ROLE db_datareader
ADD MEMBER lmacen;


-- Encargado de compras: consultar información
ALTER ROLE db_datareader
ADD MEMBER compras;

-- Administrador: administrar la base de datos
ALTER ROLE db_owner
ADD MEMBER dba;
```

Figura 10. Asignación de roles.

```
--a¿Asignar roles fijos
USE rheem;

-- Encargado de almacén: consultar información
ALTER ROLE db_datareader
ADD MEMBER almacen;

-- Encargado de compras: consultar información
ALTER ROLE db_datareader
ADD MEMBER compras;


-- Administrador: administrar la base de datos
ALTER ROLE db_owner
ADD MEMBER dba;
```

Figura 11. Roles fijos.

```
--Conceder privilegios

-- Permisos del encargado de almacén
GRANT INSERT, UPDATE
ON inventarios
TO almacen;

-- Permisos del encargado de compras
GRANT INSERT, UPDATE
ON proveedores
TO compras;

GRANT INSERT
ON Compras
TO compras;

GRANT INSERT
ON Ventas
TO compras;
```

Figura 12. Privilegios.

Se utilizaron roles fijos para evitar la creación de los roles personalizados.

10.Diseñar una estrategia de respaldo:

Respaldo FULL. (Código)

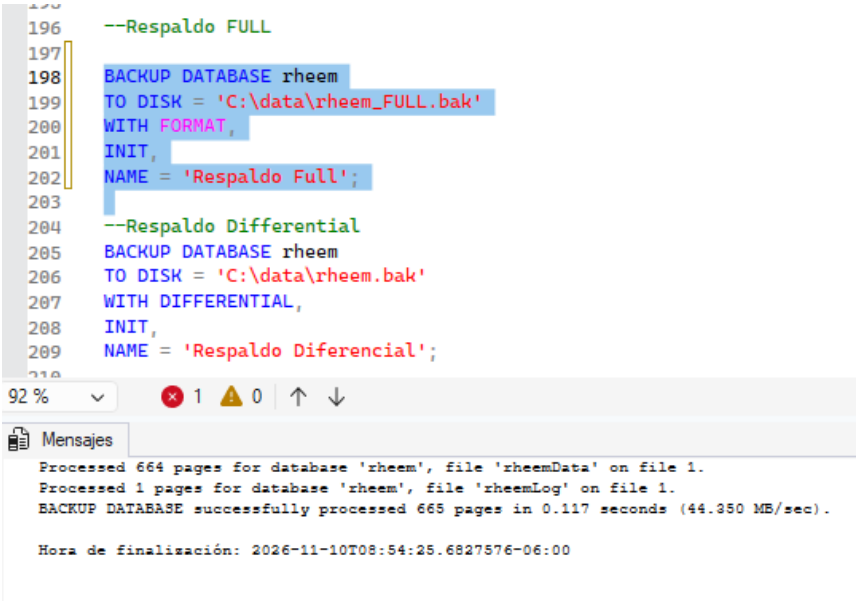


Figura 13. Full.

Respaldo DIFFERENTIAL. (Código)

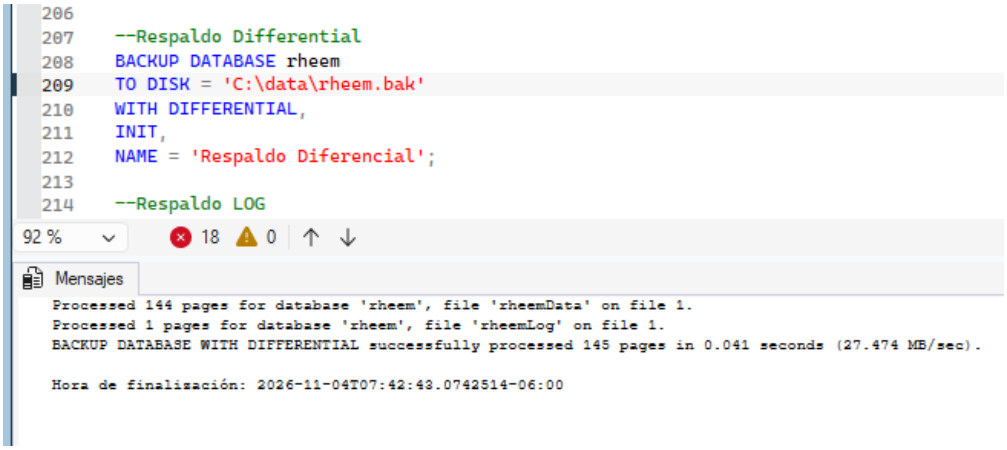


Figura 14. Differential.

Respaldo de LOG. (Código)

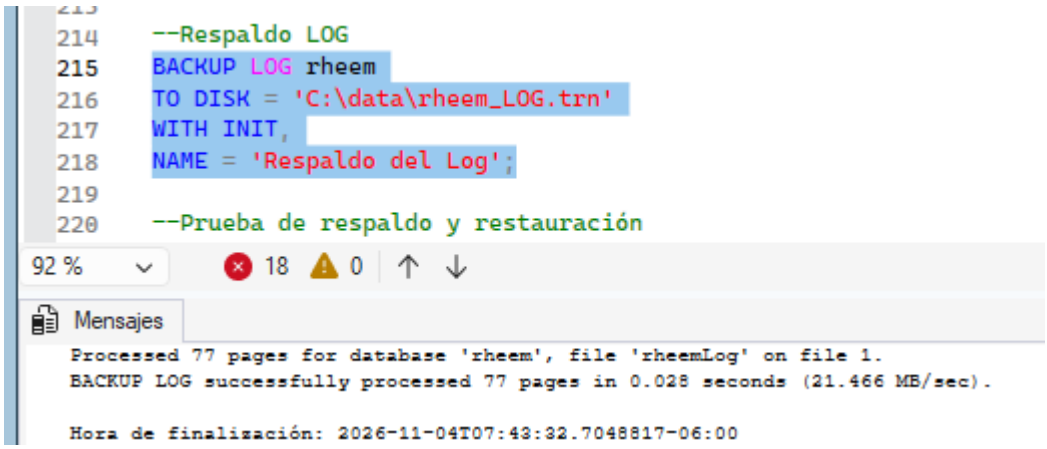


Figura 15. LOG.

Captura del respaldo exitoso



Figura 16. Respaldo.

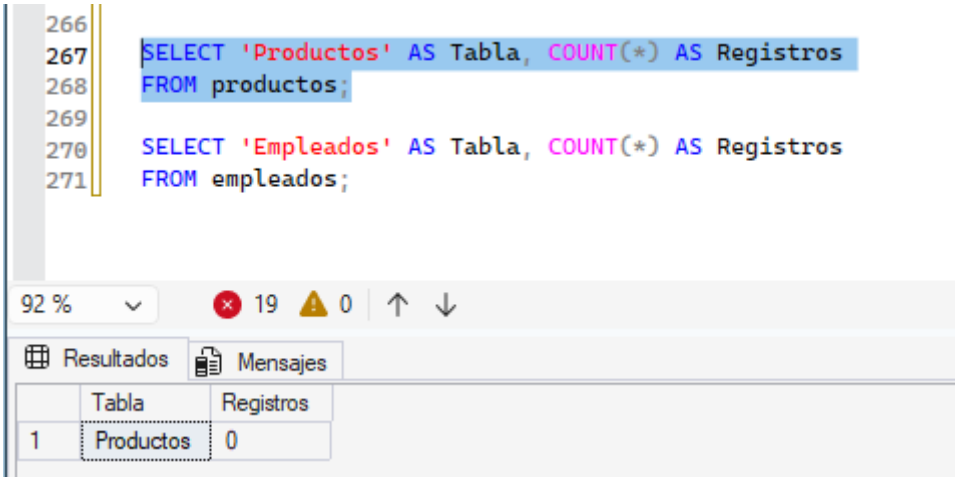


Figura 17. Respaldo.

Diferencia entre .bak y .trn:

.bak es una extensión habitual para archivos de respaldo de bases de datos, como los respaldos completos o diferenciales. .trn es una extensión convencional para respaldos del log de transacciones. SQL Server no determina el tipo de respaldo únicamente por la extensión; lo determinan el comando ejecutado y el contenido del conjunto de respaldo. Un respaldo del log requiere el modelo de recuperación FULL o BULK_LOGGED y una cadena de log válida. La secuencia de restauración debe respetar el orden: FULL, diferencial más reciente (si existe) y respaldos de log en orden cronológico.

Verificación de la base de datos después de restaurar

Después de finalizar la restauración, se debe comprobar que la base de datos se encuentre disponible y que sus tablas y registros se hayan recuperado correctamente. Para ello, se consulta la lista de tablas existentes y se revisa la cantidad de registros de cada una. También se puede ejecutar una comprobación de integridad con DBCC CHECKDB para detectar posibles errores en la base de datos.

Estas verificaciones permiten confirmar que la restauración se realizó correctamente y que la información está disponible para su uso.

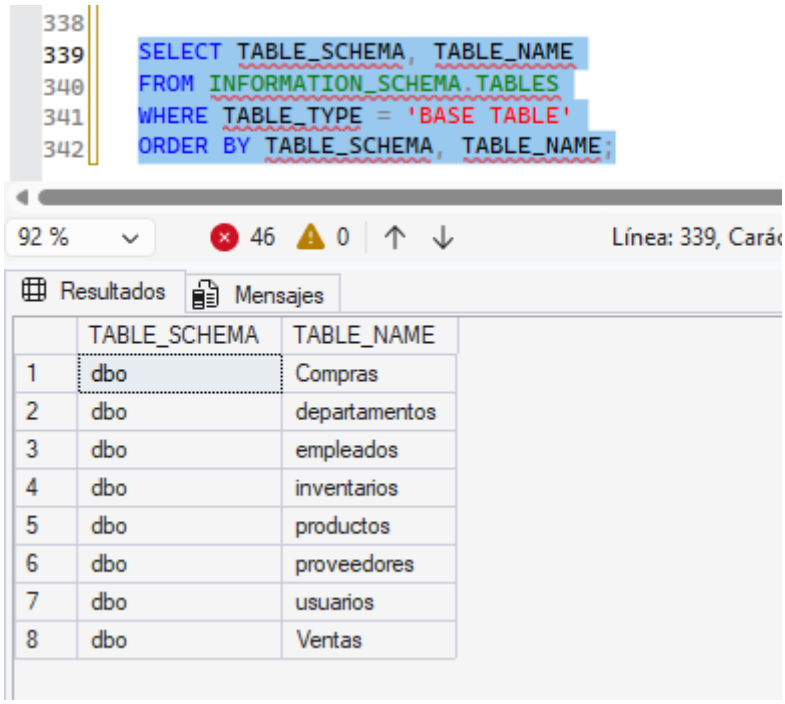


Figura 18. Verificación.

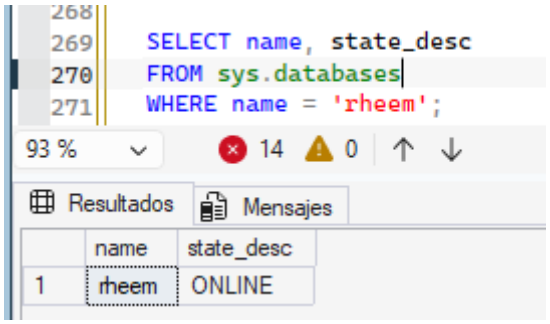


Figura 19. Verificación.

Frecuencia de cada respaldo.

Tipo	Frecuencia
Full	Cada domingo a las 00:00.
Differential	Cada 6 horas.
Log	Cada 15 minutos.

Tabla 4. Frecuencia de cada respaldo.

Ubicación donde se almacenarán.

Se propone guardar una copia temporal en un disco destinado a respaldos y enviar otra copia a un NAS independiente o almacenamiento externo protegido.

También se recomienda mantener una copia fuera de las instalaciones para reducir el riesgo de pérdida por incendio, robo o daños físicos.

Justificación de la estrategia.

La combinación de respaldos completos, diferenciales y del log permite recuperar la base de datos sin depender únicamente de una copia completa antigua.

Además, se reduce la cantidad de información que podría perderse en caso de una falla, siempre que se cumplan los horarios y se verifique el estado de los respaldos.

11.Definan el RPO de la empresa.

En la empresa se proponen perder solo 15 minutos como máximo de operaciones registradas ante una falla.

12.Definan el RTO de la empresa

En la empresa se proponen 4 horas para recuperar la base de datos y restablecer las operaciones.

13.Realicen una prueba de respaldo y restauración:

Ejecuten respaldos y expliquen el orden que utilizarían para recuperar la base de datos después de una falla.

- Restaurar el respaldo FULL utilizando **NORECOVERY.**
- Restaurar el último respaldo diferencial con **NORECOVERY.**
- Restaurar los respaldos del log en orden cronológico, también con **NORECOVERY.**
- Aplicar **RECOVERY** en el último respaldo del log para dejar disponible la base de datos.
- Verificar que las tablas y los registros se encuentren disponibles.

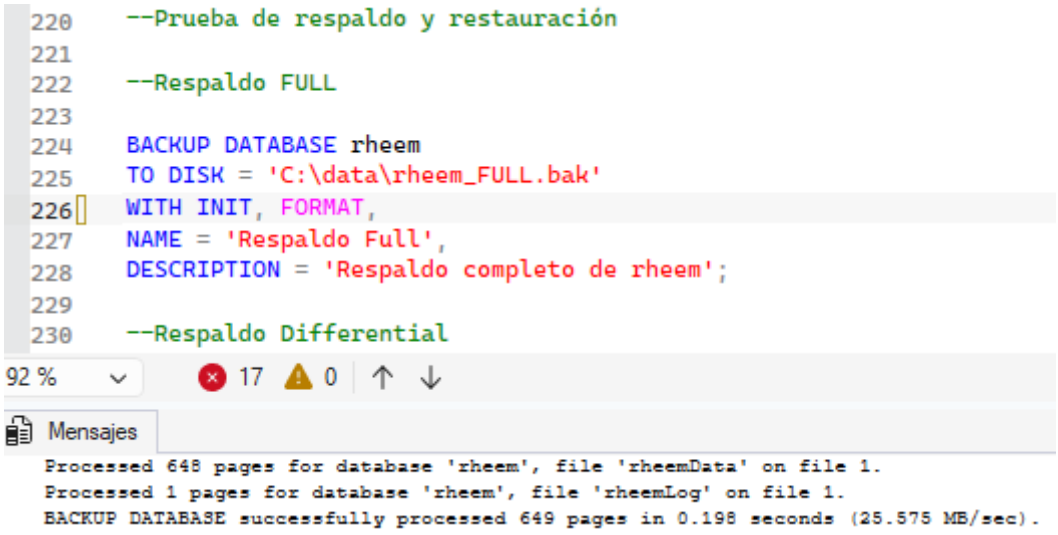


Figura 18. Códigos de restauración.

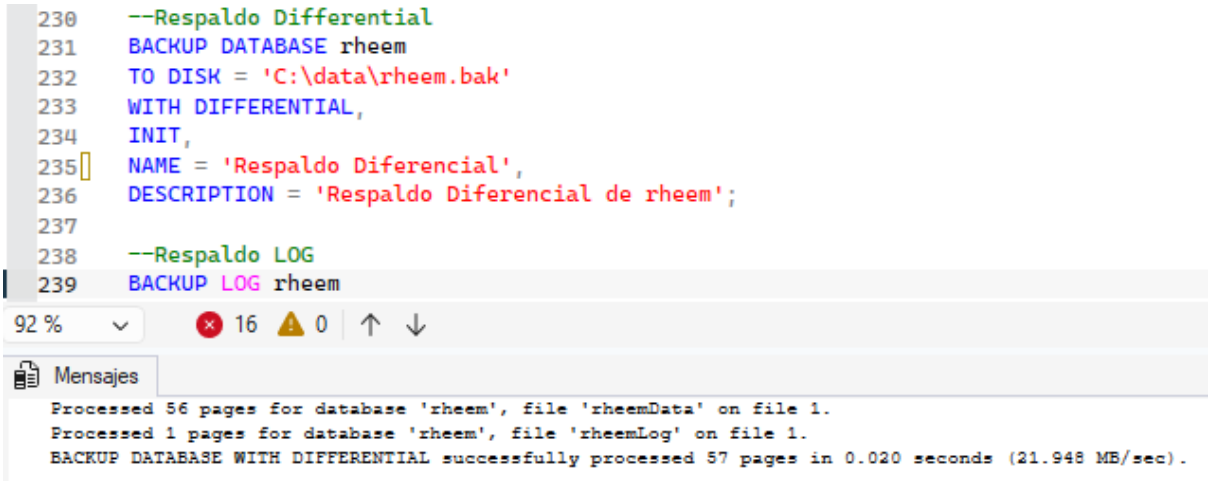


Figura 19. Códigos de restauración.

```
237
238     --Respaldo LOG
239     BACKUP LOG rheem
240     TO DISK = 'C:\data\rheem_log.bak'
241     WITH INIT,
242     NAME = 'Respaldo del Log',
243     DESCRIPTION = 'Respaldo log de rheem';
244
245     --Codigos de restauracion
246     -- Restaurar Full
```

92 % ⊗ 15 ⚠ 0

Mensajes

```
Processed 4 pages for database 'rheem', file 'rheemLog' on file 1.
BACKUP LOG successfully processed 4 pages in 0.004 seconds (7.324 MB/sec).
```

Figura 20. Código de respaldo.

```
245     --Codigos de restauracion
246     -- Restaurar Full
247     use master;
248     RESTORE DATABASE rheem
249     FROM DISK = 'C:\data\rheem_FULL.bak'
250     WITH FILE = 1,
251     REPLACE,
252     NORECOVERY,
253     STATS = 10;
```

92 % ⊗ 18 ⚠ 0

Mensajes

```
20 percent processed.
40 percent processed.
60 percent processed.
80 percent processed.
100 percent processed.
Processed 648 pages for database 'rheem', file 'rheemData' on file 1.
Processed 1 pages for database 'rheem', file 'rheemLog' on file 1.
RESTORE DATABASE successfully processed 649 pages in 0.053 seconds (95.546 MB/sec).
```

Figura 21. Restaurar.

```
253     -- Restaurar respaldo diferencial
254     RESTORE DATABASE rheem
255     FROM DISK = 'C:\data\rheem.bak'
256     WITH FILE = 1, NORECOVERY;
257
```

92 % ⊗ 15 ⚠ 0

Mensajes

```
Processed 56 pages for database 'rheem', file 'rheemData' on file 1.
Processed 1 pages for database 'rheem', file 'rheemLog' on file 1.
RESTORE DATABASE successfully processed 57 pages in 0.030 seconds (14.632 MB/sec).
```

Figura 22. Restaurar.

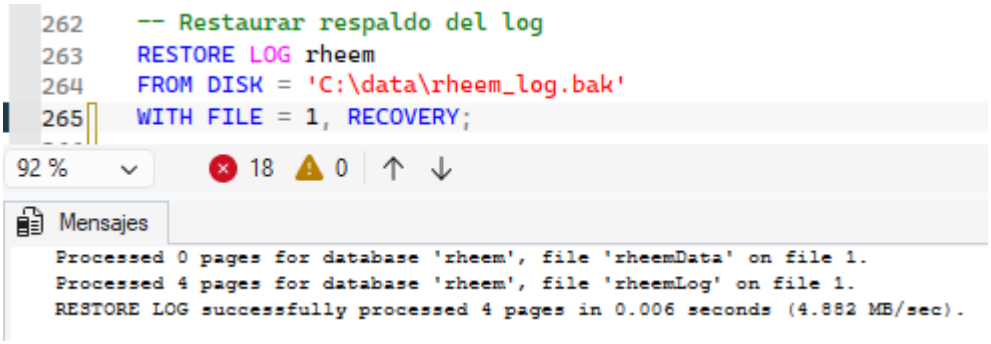


Figura 23. Restaurar.

14.Diseñen un plan de mantenimiento.

Tarea	Frecuencia	Horario	Base afectada	Justificación
Verificar respaldos	Diario	06:00	rheem	Confirmar que se generaron correctamente
Revisar espacio en discos	Diario	06:30	Servidor	Evitar falta de almacenamiento
Actualizar estadísticas	Semanal	Domingo 01:00	rheem	Mejorar la elección de planes de consulta
Revisar índices	Semanal	Domingo 02:00	rheem	Detectar fragmentación y necesidades de mantenimiento
Revisar integridad	Semanal	Domingo 03:00	rheem	Detectar errores de consistencia
Revisar usuarios y permisos	Mensual	Primer domingo, 04:00	rheem	Evitar permisos innecesarios
Probar restauración	Mensual	Primer domingo, 05:00	rheem_prueba	Comprobar que los respaldos se pueden recuperar

Tabla 5. Diseñen un plan de mantenimiento.

El plan propuesto se ejecutará en horarios de baja actividad y se registrarán fecha, responsable, resultado y evidencia. Los comandos de mantenimiento deben probarse primero en una copia de la base de datos.

1. **Verificar respaldos (diario, 06:00).** Revisar el historial de respaldos en msdb y confirmar que el archivo exista. Ejecutar RESTORE VERIFYONLY para comprobar que el medio pueda leerse. Registrar cualquier error.

2. **Revisar espacio en disco (diario, 06:30).** Consultar el espacio libre del volumen donde están los archivos MDF, LDF y respaldos. Si el espacio está por debajo del umbral definido por el administrador, documentar y escalar.

3. **Actualizar estadísticas (semanal, domingo 01:00).** Evaluar si las estadísticas están desactualizadas y actualizar las necesarias con UPDATE STATISTICS. En bases pequeñas puede evaluarse sp_updatestats, después de revisar su impacto.

4. **Revisar índices (semanal, domingo 02:00).** Consultar fragmentación y uso de índices. Reorganizar o reconstruir solo cuando los indicadores y el tamaño del índice lo justifiquen; no ejecutar reconstrucciones indiscriminadas.

5. **Comprobar integridad (semanal, domingo 03:00).** Ejecutar DBCC CHECKDB (N'rheem') WITH NO_INFOMSGS en una ventana de mantenimiento. Investigar y documentar cualquier error antes de aplicar una reparación.

6. **Revisar usuarios y permisos (mensual, primer domingo 04:00).** Comparar logins, usuarios, roles y permisos con la matriz aprobada. Retirar accesos que ya no se necesiten y guardar evidencia del cambio autorizado.

7. **Probar restauración (mensual, primer domingo 05:00).** Restaurar el último respaldo en rheem_prueba, verificar tablas y conteos, ejecutar DBCC CHECKDB y registrar cuánto tardó la recuperación. No restaurar encima de producción durante esta prueba.

15. Definan qué información deberá ser auditada.

Debido a los problemas de duplicidad, inconsistencias, modificaciones incorrectas y accesos no autorizados identificados en RHEEM, se deberá llevar un registro de las acciones importantes realizadas sobre la base de datos.

La información que deberá ser auditada será:

Información o acción	¿Qué se auditará?	Motivo
Modificación de ventas	Cambios en los datos de las operaciones de venta.	Evitar modificaciones incorrectas en la información comercial.

Modificación de productos	Cambios en nombre, precio o estado del producto	Mantener información correcta de los productos.
Movimientos de inventario	Entradas, salidas y cambios en existencias	Detectar errores o movimientos no autorizados.
Compras y ventas	Registro, modificación o eliminación de operaciones	Proteger información importante para las operaciones de la empresa.
Eliminación de registros	Usuario, fecha y registro eliminado	Poder identificar quién eliminó información.
Cambios de permisos	Creación, modificación o eliminación de permisos	Evitar que los usuarios tengan acceso a información que no necesitan.
Intentos de acceso	Accesos correctos y fallidos	Detectar intentos de acceso no autorizados.

Tabla 6. Información que auditar.

Acciones principales que serán auditadas

De acuerdo con la problemática de RHEEM, se consideran especialmente importantes las siguientes acciones:

1. Eliminación de registros: registrar cuándo se elimina información y qué usuario realizó la acción.
2. Modificación de información: registrar cambios realizados en clientes, productos, inventarios, compras y ventas.
3. Cambios de permisos: registrar cuando se asignen, modifiquen o retiren permisos a los usuarios.
4. Creación de usuarios: registrar la creación de nuevas cuentas para saber quién las creó y cuándo.
5. Intentos de acceso: registrar accesos exitosos y fallidos para detectar posibles accesos no autorizados.

Esto se relaciona con las necesidades de seguridad que ya definieron, donde indican que RHEEM debe contar con diferentes permisos, control de acceso por departamentos y auditoría de cambios importantes.

16. Implementen mecanismos de auditoría.

Auditoría a nivel de base de datos

Se propone utilizar una SQL Server Audit para registrar las acciones importantes realizadas sobre la base de datos.

La auditoría permitirá registrar información como:

- Usuario que realizó la acción.
- Fecha y hora.
- Acción realizada.
- Base de datos afectada.
- Objeto o tabla involucrada.
- Resultado de la operación.

De esta manera, será posible conocer quién realizó una determinada acción y cuándo ocurrió.

Eventos que se registrarán

La auditoría se configurará para registrar principalmente:

- Creación de usuarios.
- Cambios de permisos.
- Eliminación de registros.
- Modificaciones de información.
- Intentos de acceso.
- Acciones realizadas sobre tablas importantes.

Las tablas que tendrán mayor importancia para la auditoría serán Productos, Inventarios, Proveedores, Compras y Ventas, ya que contienen información relacionada directamente con las operaciones de RHEEM. En el proyecto se estableció que la base de datos almacena precisamente este tipo de información.

1. Definir qué eventos se auditarán: accesos fallidos, cambios de permisos, creación o eliminación de usuarios y operaciones sobre las tablas críticas.
2. Crear un destino protegido para los archivos de auditoría y comprobar que la cuenta del servicio SQL Server tenga permisos adecuados.

3. Crear la auditoría del servidor y especificar el destino del archivo. El nombre de archivo y la ruta deben adaptarse al equipo real.
4. Crear la especificación de auditoría de base de datos para las acciones que se desean registrar en la base rheem.
5. Habilitar primero la especificación y después la auditoría, siguiendo el orden correcto de ejecución.
6. Generar una prueba controlada, por ejemplo una modificación de un producto de prueba o un cambio de permiso autorizado.
7. Consultar los registros de auditoría y comprobar que aparezcan fecha/hora, usuario, acción y objeto afectado cuando el evento configurado permita esos datos.
8. Guardar capturas del código de creación, del estado habilitado y de los resultados de la consulta de auditoría.
9. Documentar la revisión periódica de los archivos, su retención y quién tiene permiso para consultarlos.

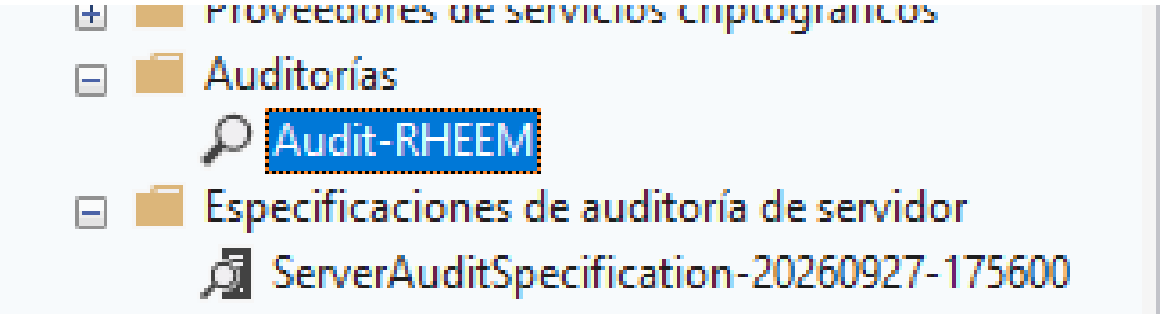


Figura 24. Prueba de auditoría.

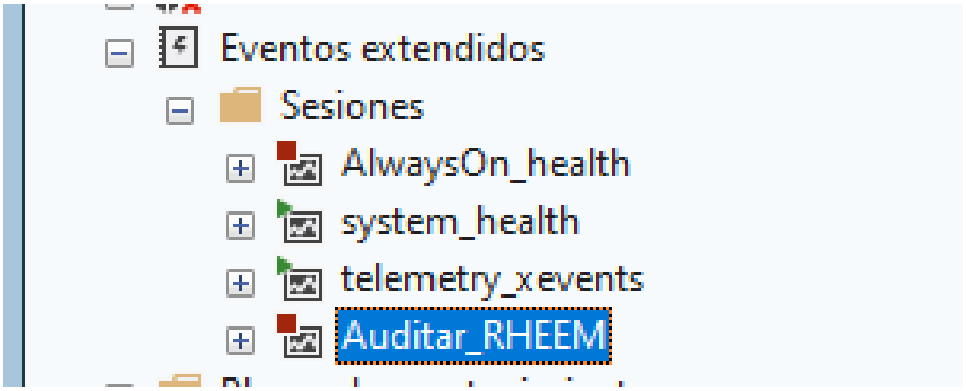


Figura 25. Prueba Evento Extendido.

Prueba de funcionamiento de la auditoría

Para comprobar el funcionamiento del mecanismo de auditoría en SQL Server, se realizó una modificación de prueba sobre la tabla Productos de la base de datos RHEEM. Se configuró SQL Server Audit para registrar las operaciones de actualización realizadas sobre esta tabla.

Posteriormente, se ejecutó una consulta mediante la función sys.fn_get_audit_file, que permitió revisar los eventos almacenados en el archivo de auditoría. En los resultados se puede identificar el usuario que realizó la operación, la fecha y hora, la tabla afectada y la instrucción ejecutada.

Esta prueba permite comprobar que la auditoría registra las modificaciones realizadas y ayuda a mantener un mayor control sobre las operaciones de la base de datos.

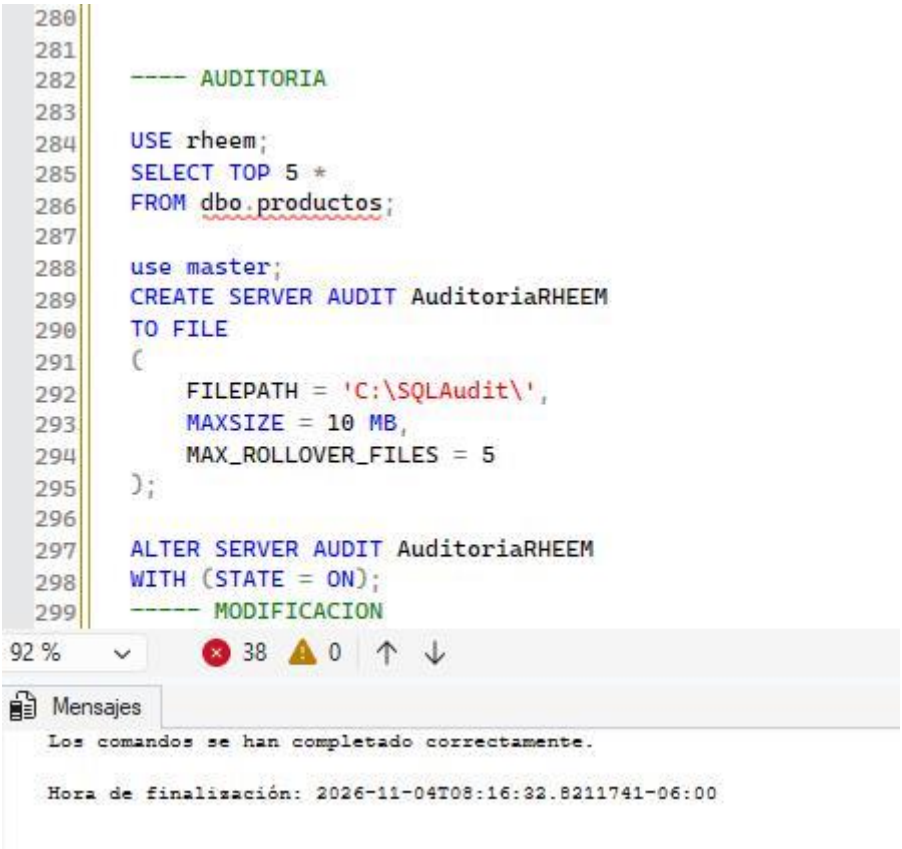


Figura 26. Configuración y habilitación del mecanismo de auditoría en SQL Server.

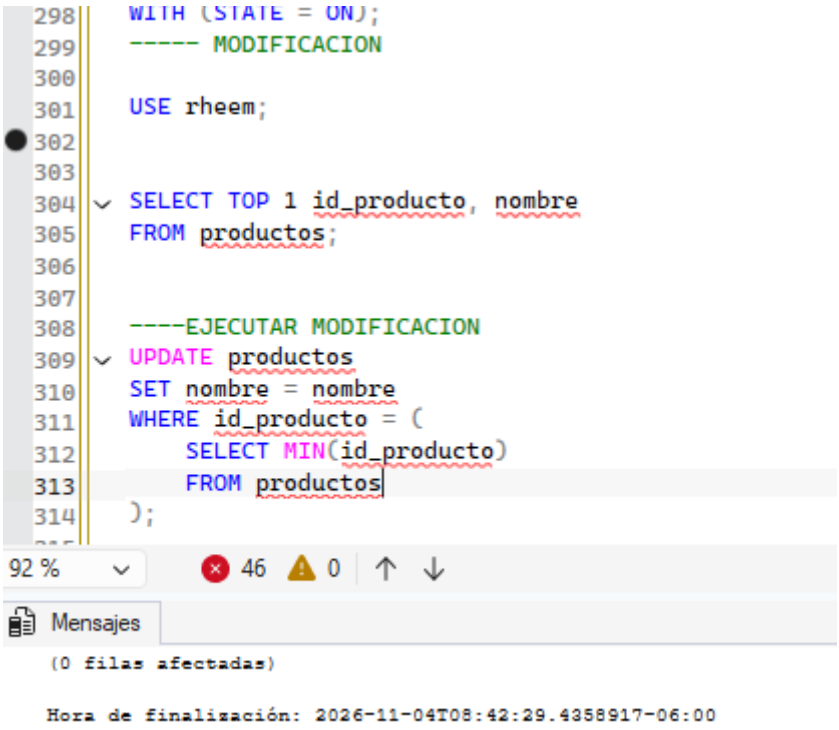


Figura 27. Ejecución de una modificación de prueba en la tabla Productos.

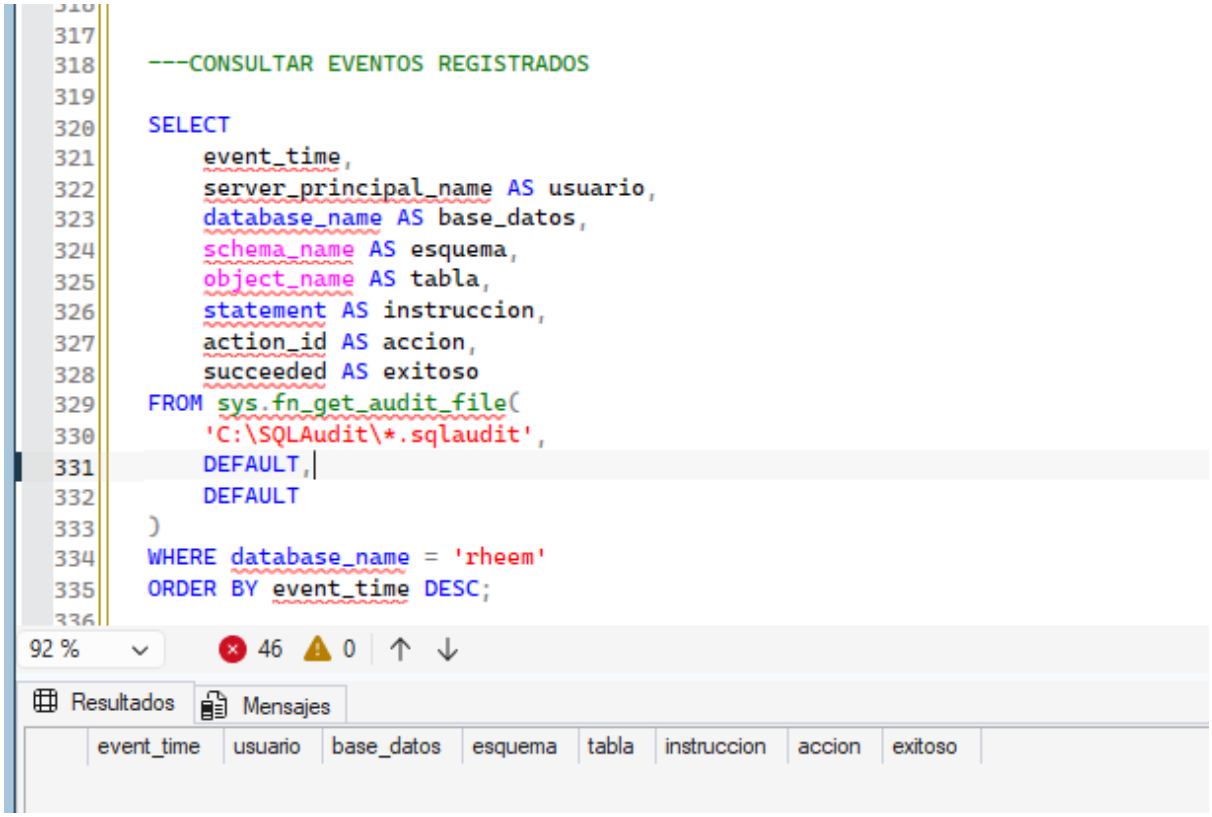


Figura 28. Consulta de los eventos registrados por la auditoría de la base de datos RHEEM.

Proyecto unidad 2

1. Problemática y diseño de datos

1.1 Problemática de la empresa

En el proyecto anterior se identificó que RHEEM presenta una problemática relacionada con la mala organización de la información en su base de datos, principalmente por la duplicidad e inconsistencia de algunos registros.

Por ejemplo, pueden existir varios registros del mismo producto o información incompleta sobre los proveedores. Además, algunos datos, como las ubicaciones de los productos en el almacén, pueden registrarse de manera incorrecta o incompleta.

Estos problemas pueden ocasionar dificultades para encontrar información, actualizar registros y generar reportes. También pueden provocar errores al tomar decisiones, debido a que los datos utilizados no siempre son confiables.

Por esta razón, en esta segunda parte del proyecto se busca analizar la calidad de los datos de RHEEM, identificar los errores presentes en sus registros y aplicar procesos para corregirlos o eliminarlos cuando sea necesario.

1.2 Diseño de los datos

Para continuar con el proyecto de la Unidad 1, se trabajará con información relacionada con las operaciones de RHEEM, especialmente con los proveedores, productos e inventarios. Estas áreas son importantes porque permiten mantener el control de la información comercial y de las existencias de la empresa.

El diseño de los datos consiste en organizar la información en tablas y campos, procurando que cada registro tenga datos completos, correctos y consistentes. Para realizar el análisis, se utilizarán archivos csv que contendrán registros de ejemplo, algunos de ellos con errores intencionales para simular los problemas de calidad identificados en el proyecto anterior.

Posteriormente, mediante programas desarrollados en Python, se revisarán los archivos para detectar registros duplicados, campos vacíos, formatos incorrectos y otros posibles errores. Una vez identificados, se determinará cuáles datos pueden corregirse y cuáles deben descartarse.

1.3 Tablas seleccionadas para el análisis

Tomando como referencia la base de datos diseñada en la Unidad 1, se consideran las siguientes tablas para el análisis de la calidad de los datos:

Tabla	Clave primaria	Relaciones	Función en la operación
Productos	id_producto	Se relaciona con Inventarios mediante id_producto.	Almacena los productos que comercializa RHEEM, incluyendo nombre, categoría y precio.
Inventarios	id_inventario	id_producto es clave foránea que referencia Productos.	Permite controlar las existencias y ubicaciones de los productos.
Proveedores	id_proveedor	Se relaciona con Compras mediante id_proveedor.	Almacena los datos de los proveedores que suministran materiales y productos.

Tabla 7. Tablas para el análisis.

Riesgos de integridad y controles aplicables

Tabla	**Tabla**	**Tabla**	**Tabla**
Riesgo identificado	Riesgo identificado	Riesgo identificado	Riesgo identificado
Control de riesgo	Control de riesgo	Control de riesgo	Control de riesgo
Código del control	Código del control	Código del control	Código del control

Tabla 8. Riesgos de integridad.

Los códigos de control se asignaron para identificar cada medida de prevención dentro del proyecto. Su finalidad es facilitar la identificación de los riesgos y mantener un control sobre la calidad de la información almacenada en las tablas de RHEEM.

1.4 Reglas de calidad de los datos

Para detectar los errores en los archivos csv, se establecerán algunas reglas que permitan verificar si la información cumple con las condiciones necesarias.

- Productos: el nombre debe estar completo y el precio debe ser un valor numérico válido, sin cantidades negativas.
- Inventarios: las cantidades deben ser numéricas y no pueden ser negativas. Además, cada registro debe corresponder a un producto existente.

- Proveedores: el nombre debe estar completo y los datos de contacto deben tener un formato válido.
- Todas las tablas: se revisará que no existan registros duplicados ni campos obligatorios vacíos.

Estas reglas ayudarán a identificar los registros que necesitan corregirse o eliminarse antes de importar la información a SQL Server.

1.5 Importancia de mejorar la calidad de los datos

Mejorar la calidad de los datos permitirá que RHEEM tenga información más confiable y organizada. Al eliminar duplicados, corregir errores y validar los registros, será más sencillo consultar los datos, mantener actualizado el inventario y generar reportes útiles para los diferentes departamentos de la empresa.

Además, este proceso dará continuidad al trabajo de la Unidad 1, en el que se destacó la importancia de mantener la información organizada, protegida y disponible para apoyar las operaciones y la toma de decisiones.

2. Creación de los archivos csv

Para realizar el análisis de calidad de datos de RHEEM, se crearán tres archivos csv correspondientes a las tablas Productos, Inventarios y Proveedores de la base de datos.

Cada archivo contendrá 50 registros de ejemplo. Algunos datos tendrán errores intencionales, como campos vacíos, registros duplicados, precios incorrectos, existencias negativas y formatos inválidos en teléfonos o correos electrónicos.

Los archivos se utilizarán para identificar problemas de calidad de datos mediante programas desarrollados en Python. Posteriormente, se aplicarán procesos de limpieza para corregir los errores que sea posible solucionar y eliminar los registros que no cumplan con las reglas establecidas.

Los archivos generados serán productos.csv, inventarios.csv y proveedores.csv, los cuales permitirán comparar la información original con los resultados obtenidos después de la limpieza.

2.1. Código Python

```python
#PROYECTO 2: INTEGRANTES
#MONICA FERNANDA ALFARO RANGEL Y DIANA MARITZA MENDOZA COSTILLA


import csv
import random
from faker import Faker

fake = Faker("es_MX")
random.seed(42)
Faker.seed(42)

# 1. GENERAR PRODUCTOS


productos = []
nombres_productos = [
    "Aire acondicionado", "Calentador de agua",
    "Termostato", "Minisplit", "Ventilador",
    "Bomba de calor", "Filtro de aire",
    "Control remoto", "Radiador", "Condensador"
]
```

Figura 29. Código Python.

```python
categorias = [
    "Climatización", "Calefacción",
    "Agua caliente", "Accesorios"
]

for i in range(1, 51):
    productos.append({
        "id_producto": i,
        "nombre": random.choice(nombres_productos) + f" Modelo {i}",
        "categoria": random.choice(categorias),
        "precio": round(random.uniform(500, 15000), 2),
        "activo": random.choice([0, 1])
    })

# Errores intencionales
productos[4] = productos[3].copy()  # Duplicado
productos[9]["precio"] = -250                    # Precio negativo
productos[14]["nombre"] = ""                     # Nombre vacío
productos[19]["precio"] = "precio desconocido"   # Precio inválido
productos[24]["categoria"] = ""                  # Categoría vacía
```

Figura 30. Código Python.

productos.csv

Nombres duplicados, nombre vacío, precio negativo y precio no numérico.

¿Por qué seleccionamos estos errores?

Se eligieron nombres duplicados, campos vacíos, precios negativos y precios con texto porque son errores que pueden afectar la organización del catálogo y la confiabilidad de la información.

Ejemplo en producción: un empleado registra dos veces el mismo modelo de aire acondicionado, captura un precio negativo por equivocación o deja el nombre del producto vacío. Esto puede provocar confusiones al consultar el catálogo, calcular costos o realizar una venta.

```python
with open("productos.csv", "w", newline="", encoding="utf-8-sig") as archivo:
    campos = ["id_producto", "nombre", "categoria", "precio", "activo"]
    escritor = csv.DictWriter(archivo, fieldnames=campos)
    escritor.writeheader()
    escritor.writerows(productos)


# 2. GENERAR INVENTARIOS


inventarios = []

for i in range(1, 51):
    inventarios.append({
        "id_inventario": i,
        "id_producto": i,
        "existencia": random.randint(0, 100),
        "ubicacion": f"Almacen {random.randint(1, 5)}"
    })
```

Figura 31. Código Python.

```python
# Errores intencionales
inventarios[5]["existencia"] = -10
inventarios[9]["ubicacion"] = ""
inventarios[14]["id_producto"] = 999  # producto inexistente
inventarios[19]["existencia"] = "muchas"
inventarios[24] = inventarios[23].copy()  # duplicado

with open("inventarios.csv", "w", newline="", encoding="utf-8-sig") as archivo:
    campos = ["id_inventario", "id_producto", "existencia", "ubicacion"]
    escritor = csv.DictWriter(archivo, fieldnames=campos)
    escritor.writeheader()
    escritor.writerows(inventarios)
```

Figura 32. Código Python.

inventarios.csv

Existencia negativa, ubicación vacía, producto inexistente y existencia no numérica.

¿Por qué seleccionamos estos errores?

Se eligieron existencias negativas, ubicaciones vacías, cantidades no numéricas y referencias a productos inexistentes porque pueden impedir que el inventario represente las existencias reales de la empresa.

Ejemplo en producción: un trabajador registra que hay menos diez calentadores en almacén, escribe “muchas” en lugar de una cantidad o asigna un producto a una ubicación que no existe. Esto puede ocasionar diferencias entre el inventario registrado y los productos disponibles físicamente.

```python
# 3. GENERAR PROVEEDORES


proveedores = []

for i in range(1, 51):
    proveedores.append({
        "id_proveedor": i,
        "nombre": fake.company()[:150],
        "telefono": fake.numerify("##########"),
        "correo": fake.email()
    })

# Errores intencionales
proveedores[4] = proveedores[3].copy()  # Duplicado
proveedores[9]["telefono"] = "teléfono incorrecto"
proveedores[14]["correo"] = "correo-sin-formato"
proveedores[19]["nombre"] = ""
proveedores[24]["correo"] = ""
```

Figura 33. Código Python.

proveedores.csv

Nombre duplicado, nombre vacío, teléfono inválido y correo incorrecto.

¿Por qué seleccionamos estos errores?

Se eligieron nombres duplicados, teléfonos incorrectos, correos sin formato válido y campos vacíos porque dificultan la identificación y comunicación con los proveedores.

Ejemplo en producción: un empleado registra dos veces al mismo proveedor o captura mal su teléfono. Cuando el departamento de compras necesita solicitar materiales para fabricar equipos de climatización, podría comunicarse con el contacto equivocado o no lograr localizarlo.

```python
with open("proveedores.csv", "w", newline="", encoding="utf-8-sig") as archivo:
    campos = ["id_proveedor", "nombre", "telefono", "correo"]
    escritor = csv.DictWriter(archivo, fieldnames=campos)
    escritor.writeheader()
    escritor.writerows(proveedores)


print("Archivos CSV creados correctamente.")
print("productos.csv: 50 registros")
print("inventarios.csv: 50 registros")
print("proveedores.csv: 50 registros")
print("Total: 150 registros de ejemplo.")
```

Figura 34. Código Python.

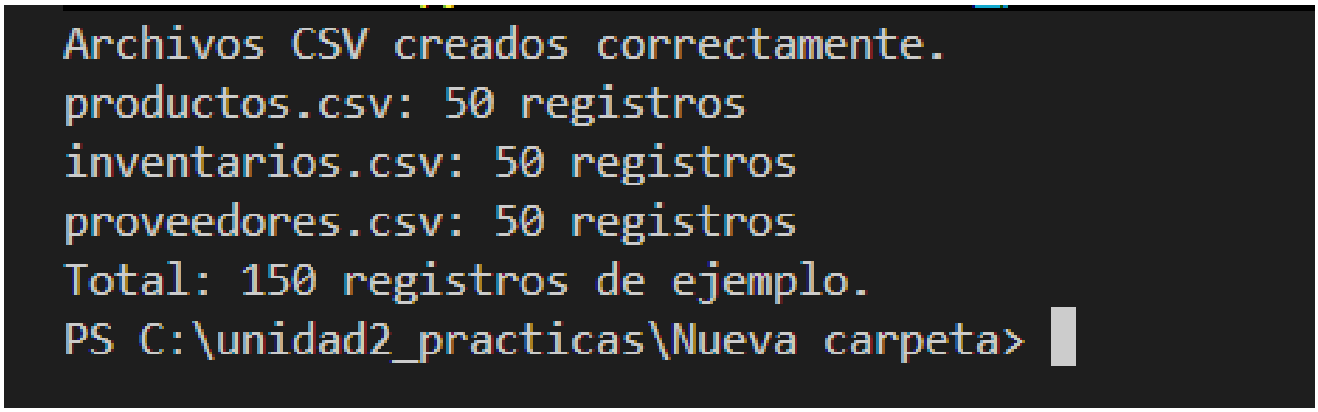


Figura 35. Resultados de código.

2.2 Diagnóstico de la calidad de los datos.

El diagnóstico de la calidad de los datos tiene como objetivo determinar si la información registrada en los archivos csv de RHEEM es confiable y suficiente para apoyar las decisiones relacionadas con el control de productos, inventarios y proveedores.

Para realizar este diagnóstico, se plantean las siguientes 7 preguntas de investigación:

1. Calidad general: ¿Qué porcentaje de los registros de productos, inventarios y proveedores cumple con las reglas de calidad establecidas?

Para determinar la calidad general de los datos, se revisaron los archivos productos.csv, inventarios.csv y proveedores.csv mediante las reglas establecidas. Se identificaron errores como nombres vacíos, registros duplicados, precios negativos, existencias incorrectas y datos de contacto inválidos. Por lo tanto, no todos los registros cumplen inicialmente con las condiciones de calidad. Para conocer el porcentaje exacto de registros aptos, es necesario

considerar los resultados obtenidos después de ejecutar los programas de validación y limpieza en Python.

2. Consistencia: ¿Existen registros duplicados o datos que presentan diferencias en la forma de identificar productos y proveedores?
Sí. Durante la generación de los archivos se incluyeron intencionalmente registros duplicados en productos.csv y proveedores.csv. Estos errores pueden provocar que un mismo producto o proveedor aparezca más de una vez, dificultando su identificación y administración. Por esta razón, se estableció la revisión de duplicados como parte del proceso de limpieza para evitar que la información repetida afecte las consultas y reportes de RHEEM.

3. Integridad: ¿Los registros contienen todos los campos necesarios para identificar correctamente los productos, las existencias y los proveedores?
No todos los registros cumplen con esta condición. Se incluyeron campos vacíos, como nombres de productos, nombres de proveedores y ubicaciones de inventario. La ausencia de esta información dificulta identificar correctamente los registros y puede ocasionar problemas al consultar los datos. Por ello, se establecieron reglas para identificar los campos obligatorios y separar aquellos registros que no pueden corregirse de manera confiable.

4. Validez: ¿Los precios, las existencias, los teléfonos y los correos electrónicos cumplen con los formatos y valores permitidos?
Inicialmente, no todos los datos cumplen con los formatos y valores establecidos. En productos.csv se incluyeron precios negativos y valores no numéricos; en inventarios.csv se agregaron existencias negativas y cantidades no numéricas; mientras que en proveedores.csv se incluyeron teléfonos y correos electrónicos con formatos incorrectos. Estos errores deben detectarse mediante validaciones en Python para corregirlos cuando exista información confiable o separarlos para su revisión.

5. Relación entre columnas: ¿Los identificadores de productos registrados en el archivo de inventarios corresponden a productos existentes en el catálogo?
No todos los registros cumplen con esta condición, debido a que se incluyeron intencionalmente referencias a productos inexistentes en inventarios.csv. Esto representa un problema de integridad referencial, porque una existencia podría quedar asociada a un producto

que no se encuentra registrado en el catálogo. Para evitarlo, se comparan los identificadores de ambos archivos y se identifican las referencias que no tienen correspondencia en productos.csv.

6. Confiabilidad del inventario: ¿Las cantidades registradas son números enteros iguales o mayores que cero y las ubicaciones están especificadas?

Columna	Obligatoriedad	Tipo / Rango Aceptable	Regla de Negocio / Validación
id_producto	Obligatorio	Clave primaria / Numérico entero > 0	Único en el archivo; identifica unívocamente al producto.
nombre	Obligatorio	Texto (hasta 150 caracteres)	No debe estar vacío ni contener solo espacios.
categoria	Opcional	Texto	Si se especifica, debe corresponder a una categoría válida.
precio	Obligatorio	Numérico decimal > 0	Debe ser mayor a cero; no se permiten valores negativos o texto.
activo	Obligatorio	Booleano (1 o 0)	Valor por defecto 1.

Inicialmente, no todos los registros de inventarios.csv cumplen con estas condiciones. Se incluyeron cantidades negativas, valores no numéricos y ubicaciones vacías. Estos errores pueden ocasionar diferencias entre las existencias registradas y las cantidades reales de los productos almacenados. Por ello, se validan los tipos de datos, los rangos permitidos y la obligatoriedad de la ubicación para mejorar la confiabilidad del inventario.

7. Utilidad para la toma de decisiones: ¿La calidad de los datos disponibles permite generar información confiable para administrar productos, controlar inventarios y contactar proveedores?

La información original necesita un proceso de limpieza antes de utilizarse para generar reportes y apoyar la toma de decisiones. Los registros duplicados, los precios incorrectos, las existencias negativas y los datos de contacto inválidos pueden afectar las consultas y provocar resultados poco confiables. Después de aplicar las reglas de negocio y el proceso ETL, los archivos depurados pueden utilizarse para realizar consultas en SQL Server y obtener información más organizada sobre los productos, inventarios y proveedores de RHEEM.

2.3 Establece las reglas de negocio concretas en cada columna por cada archivo (puede ser una tabla): rangos aceptables, obligatoriedad, consistencia entre campos y relaciones entre archivos.

Reglas de negocio de productos.csv

Tabla 9. Reglas de negocio.

Reglas de negocio de inventarios.csv

Columna	Obligatoriedad	Tipo / Rango Aceptable	Regla de Negocio / Validación
id_inventario	Obligatorio	Numérico entero > 0	Clave primaria única de cada registro de almacén.
id_producto	Obligatorio	Numérico entero > 0	Clave foránea; debe existir obligatoriamente en productos.csv.
existencia	Obligatorio	Numérico entero ≥ 0	No se permiten cantidades negativas ni valores no numéricos.
ubicacion	Obligatorio	Texto	Debe indicar el almacén o posición física; no puede estar vacío

Tabla 10. Reglas de negocio.

3. Reglas de negocio de proveedores.csv

Columna	Obligatoriedad	Tipo / Rango Aceptable	Regla de Negocio / Validación
id_proveedor	Obligatorio	Numérico entero > 0	Clave primaria única del proveedor.
nombre	Obligatorio	Texto	No debe estar vacío ni repetirse entre distintos proveedores.
telefono	Opcional	Cadena numérica (10 dígitos)	Formato telefónico válido; sin letras ni símbolos extraños.
correo	Opcional	Formato usuario@dominio.com	Debe cumplir con la estructura válida de correo electrónico.

Tabla 11. Reglas de negocio.

2.4 Limpieza y transformación de datos

Después de analizar los archivos csv mediante Python, se realizará un proceso de limpieza para mejorar la calidad de la información. Este procedimiento consiste en revisar los registros

identificados con errores, corregir los datos cuando sea posible y descartar aquellos que no puedan recuperarse de manera confiable.
En el archivo de productos se revisarán los nombres, las categorías y los precios; en inventarios se verificarán las existencias, las ubicaciones y las referencias a los productos; y en proveedores se validarán los nombres y los datos de contacto.
Los archivos originales se conservarán para comparar los resultados. Los datos corregidos se guardarán en nuevos archivos csv, lo que permitirá comprobar si disminuyeron los errores antes de importar la información a SQL Server.

2.5 Explica la importancia de ETL

El proceso ETL es importante porque permite extraer información de diferentes archivos, transformarla mediante reglas de calidad y cargarla en una base de datos organizada. En el caso de RHEEM, permite detectar y corregir errores antes de integrar la información en SQL Server. De esta manera, se evita que los registros incorrectos afecten el control de productos, las existencias y la información de los proveedores.

Error encontrado	Columna	Registros afectados	Operación realizada	Posible consecuencia
Registros duplicados exactos	Todas las columnas	Productos, inventarios y proveedores	Eliminar duplicados conservando un registro.	Conteos y reportes incorrectos.
Precio negativo o no numérico	precio	Productos	Convertir a numérico y descartar valores inválidos.	Cálculos de precios incorrectos.
Nombre vacío	nombre	Productos y proveedores	Identificar y separar los registros.	Dificultad para identificar los registros.
Existencia negativa o no numérica	existencia	Inventarios	Validar el tipo de dato y el rango permitido.	Diferencias entre inventario real y registrado.
Producto inexistente	id_producto	Inventarios	Comprobar la relación con productos.csv y descartar referencias rotas.	Existencias asociadas a productos inexistentes.
Ubicación vacía	ubicacion	Inventarios	Separar registros sin ubicación válida.	Dificultad para localizar productos.
Teléfono inválido	telefono	Proveedores	Validar que tenga diez	Problemas de comunicación.

			dígitos numéricos.	
Correo inválido o vacío	correo	Proveedores	Validar el formato y separar registros incorrectos.	Imposibilidad de contactar al proveedor.

Tabla 12. ETL.

2.6 Validación y comparación antes/después:

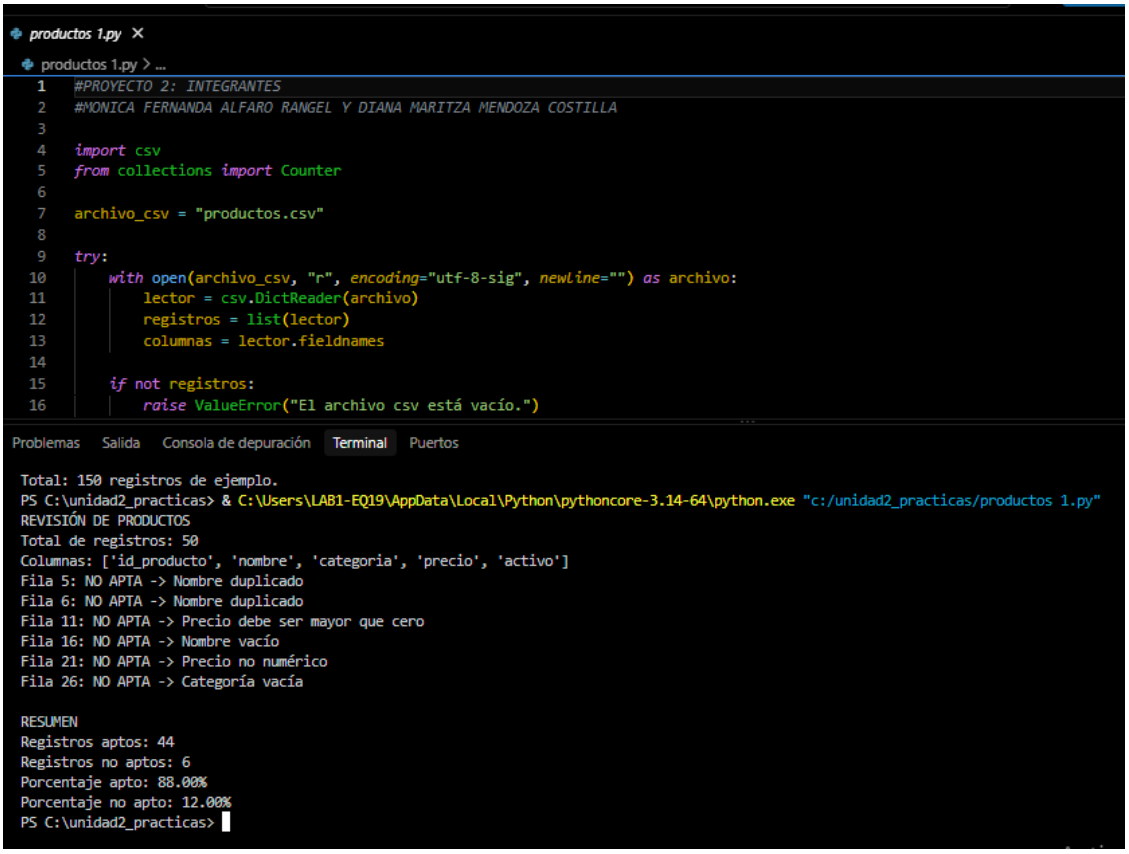
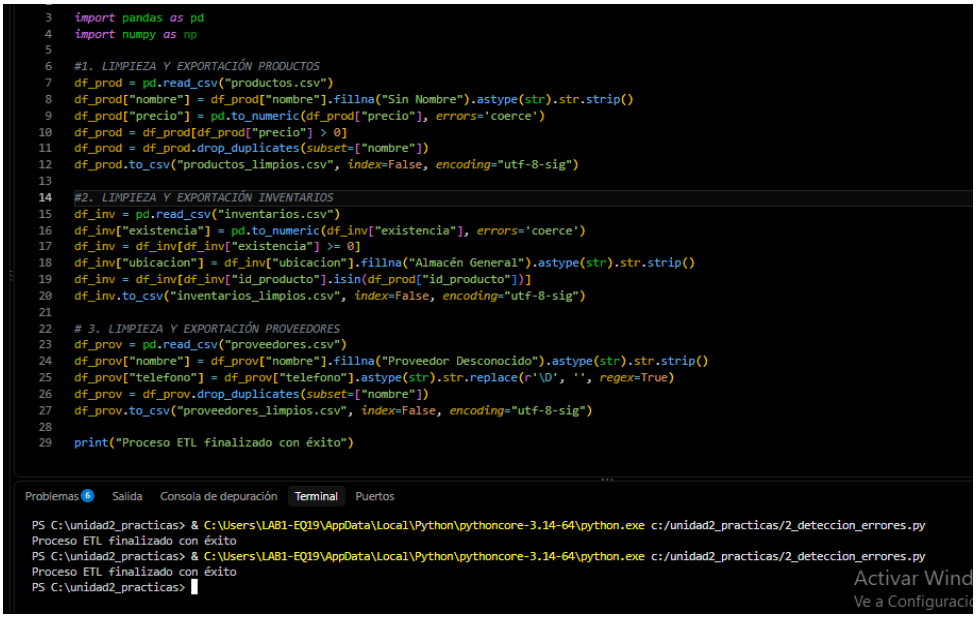


Figura 36. Validación

2.7 Exportación de datos y reintegración

- Exporta los datos a csv, sin incluir la columna de índice de Pandas



```python
import pandas as pd
import numpy as np

#1. LIMPIEZA Y EXPORTACIÓN PRODUCTOS
df_prod = pd.read_csv("productos.csv")
df_prod["nombre"] = df_prod["nombre"].fillna("Sin Nombre").astype(str).str.strip()
df_prod["precio"] = pd.to_numeric(df_prod["precio"], errors='coerce')
df_prod = df_prod[df_prod["precio"] > 0]
df_prod = df_prod.drop_duplicates(subset=["nombre"])
df_prod.to_csv("productos_limpios.csv", index=False, encoding="utf-8-sig")

#2. LIMPIEZA Y EXPORTACIÓN INVENTARIOS
df_inv = pd.read_csv("inventarios.csv")
df_inv["existencia"] = pd.to_numeric(df_inv["existencia"], errors='coerce')
df_inv = df_inv[df_inv["existencia"] >= 0]
df_inv["ubicacion"] = df_inv["ubicacion"].fillna("Almacén General").astype(str).str.strip()
df_inv = df_inv[df_inv["id_producto"].isin(df_prod["id_producto"])]
df_inv.to_csv("inventarios_limpios.csv", index=False, encoding="utf-8-sig")

# 3. LIMPIEZA Y EXPORTACIÓN PROVEEDORES
df_prov = pd.read_csv("proveedores.csv")
df_prov["nombre"] = df_prov["nombre"].fillna("Proveedor Desconocido").astype(str).str.strip()
df_prov["telefono"] = df_prov["telefono"].astype(str).str.replace(r'\D', '', regex=True)
df_prov = df_prov.drop_duplicates(subset=["nombre"])
df_prov.to_csv("proveedores_limpios.csv", index=False, encoding="utf-8-sig")

print("Proceso ETL finalizado con éxito")
```

Problemas 6 Salida Consola de depuración Terminal Puertos

```
PS C:\unidad2_practicas> & C:\Users\LAB1-EQ19\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/unidad2_practicas/2_deteccion_errores.py
Proceso ETL finalizado con éxito
PS C:\unidad2_practicas> & C:\Users\LAB1-EQ19\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/unidad2_practicas/2_deteccion_errores.py
Proceso ETL finalizado con éxito
PS C:\unidad2_practicas>
```

Activar Wind
Ve a Configuraci

Figura 37. Exportación

```python
# PROYECTO 2: IMPORTACION DE DATOS A SQL SERVER
# MONICA FERNANDA ALFARO RANGEL
# DIANA MARITZA MENDOZA COSTILLA

import pandas as pd
import pyodbc
from sqlalchemy import create_engine, URL
from sqlalchemy.exc import SQLAlchemyError

# 1. CONFIGURAR SERVIDOR Y BASE DE DATOS
servidor = r"localhost\LAB3-EQ20"
base_datos = "rheem"

print("CONTROLADORES DISPONIBLES:")
print(pyodbc.drivers())

# 2. CADENA DE CONEXION
cadena_conexion = (
    "DRIVER={ODBC Driver 17 for SQL Server};"
    f"SERVER={servidor};"
    f"DATABASE={base_datos};"
    "Trusted_Connection=yes;"
)

# 3. CREAR URL Y MOTOR
url = URL.create(
    "mssql+pyodbc",
    query={"odbc_connect": cadena_conexion}
)
```

Figura 38. Exportación.

```
motor = create_engine(url)

try:

    # 4. LEER LOS ARCHIVOS LIMPIOS
    df_prod = pd.read_csv(
        "productos_limpios.csv",
        encoding="utf-8-sig"
    )

    df_prov = pd.read_csv(
        "proveedores_limpios.csv",
        encoding="utf-8-sig",
        dtype={"telefono": "string", "correo": "string"}
    )

    df_inv = pd.read_csv(
        "inventarios_limpios.csv",
        encoding="utf-8-sig"
    )

    print("\nARCHIVOS CSV LEIDOS CORRECTAMENTE")
```

Figura 39. Exportación.

```
# 5. VERIFICAR PROVEEDORES
# NO IMPORTAR REGISTROS CON CAMPOS OBLIGATORIOS VACIOS

df_prov["telefono"] = (
    df_prov["telefono"]
    .str.replace(r"\D", "", regex=True)
)

proveedores_invalidos = df_prov[
    df_prov["telefono"].isna()
    | (df_prov["telefono"] == "")
    | df_prov["correo"].isna()
    | (df_prov["correo"].str.strip() == "")
]

print("\nPROVEEDORES EXCLUIDOS:")
print(proveedores_invalidos[["id_proveedor", "nombre"]])

df_prov = df_prov.drop(proveedores_invalidos.index)

# 6. VERIFICAR RELACION PRODUCTOS-INVENTARIOS
ids_productos = set(df_prod["id_producto"])

ids_inventarios = set(df_inv["id_producto"])

if not ids_inventarios.issubset(ids_productos):
    raise ValueError(
        "Existen inventarios con productos inexistentes."
    )
```

Figura 40. Exportación.

```python
# 7. CONECTAR CON SQL SERVER
with motor.begin() as conexion:

    print("\nCONEXION EXITOSA CON SQL SERVER")

    # 8. COMPROBAR QUE LAS TABLAS ESTEN VACIAS
    tablas = ["Productos", "Proveedores", "Inventarios"]

    for tabla in tablas:
        total = conexion.exec_driver_sql(
            f"SELECT COUNT(*) FROM dbo.{tabla}"
        ).scalar_one()

        if total > 0:
            raise ValueError(
                f"La tabla {tabla} ya contiene registros. "
                "Se cancelo la importacion para evitar duplicados."
            )

    # 9. FUNCION PARA INSERTAR CONSERVANDO LOS IDENTITY
    def importar_tabla(df, tabla):

        conexion.exec_driver_sql(
            f"SET IDENTITY_INSERT dbo.{tabla} ON"
        )

        try:
            df.to_sql(
                tabla,
                con=conexion,
                schema="dbo",
                if_exists="append",
                index=False
            )
        finally:
            conexion.exec_driver_sql(
                f"SET IDENTITY_INSERT dbo.{tabla} OFF"
            )

        print(
            f"{tabla}: {len(df)} registros importados correctamente."
        )
```

Figura 41. Exportación.

```
        # 10. IMPORTAR PRODUCTOS
        importar_tabla(df_prod, "Productos")

        # 11. IMPORTAR PROVEEDORES
        importar_tabla(df_prov, "Proveedores")

        # 12. IMPORTAR INVENTARIOS
        importar_tabla(df_inv, "Inventarios")

        print("\nIMPORTACION FINALIZADA EXITOSAMENTE")

except FileNotFoundError as error:
    print("No se encontro el archivo:", error.filename)

except (SQLAlchemyError, ValueError) as error:
    print("ERROR DURANTE LA IMPORTACION:", error)

except Exception as error:
    print("ERROR:", error)

finally:
    motor.dispose()
```

Figura 42. Exportación.

- Importa los datos a SQL SERVER (integra capturas y describe el proceso)

  Inventarios_limpios.csv:

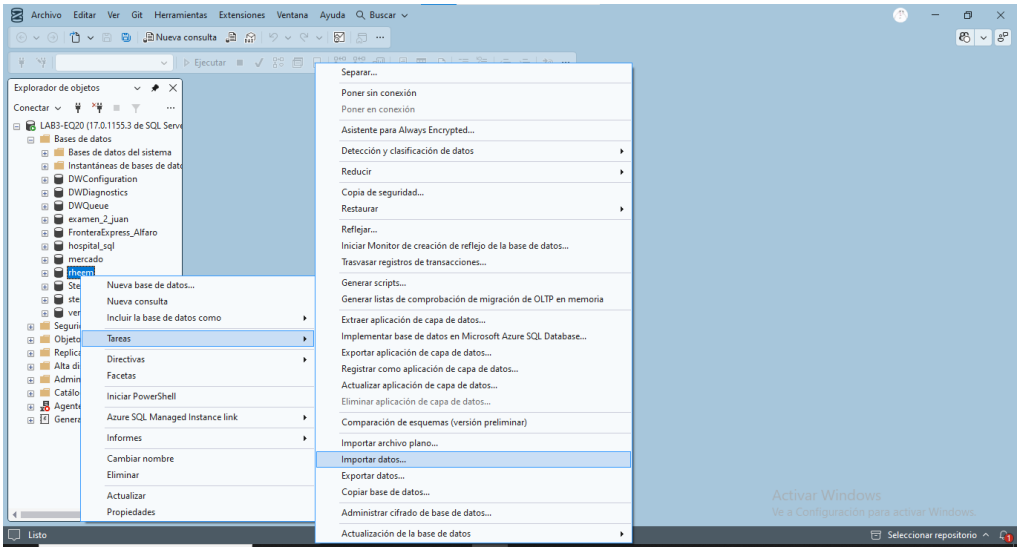


Figura 43. Importación.

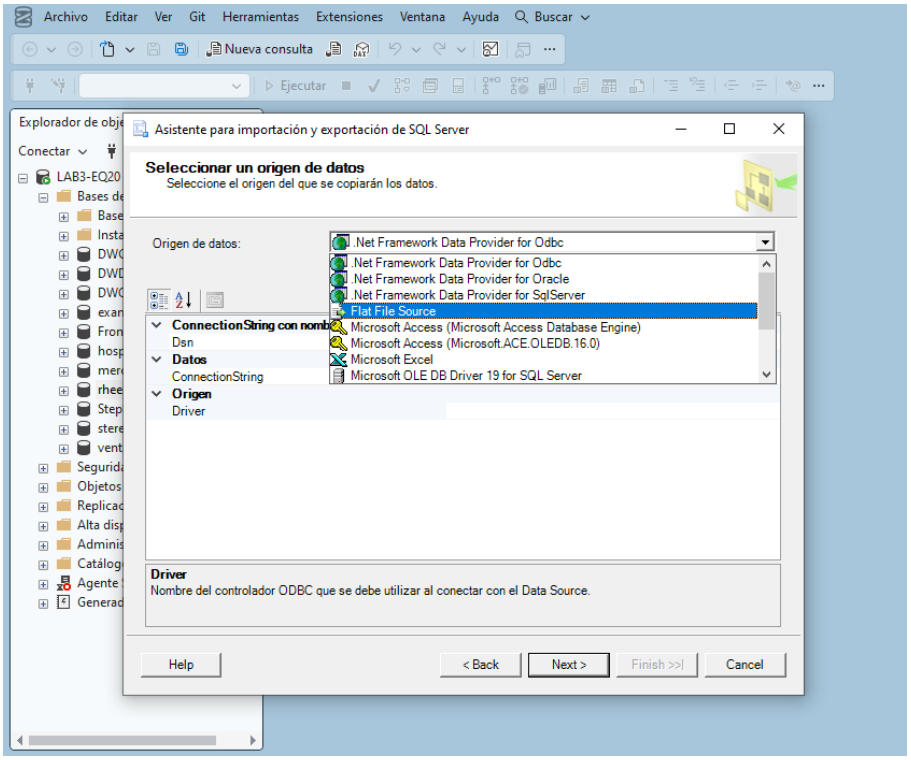


Figura 44. Importación.

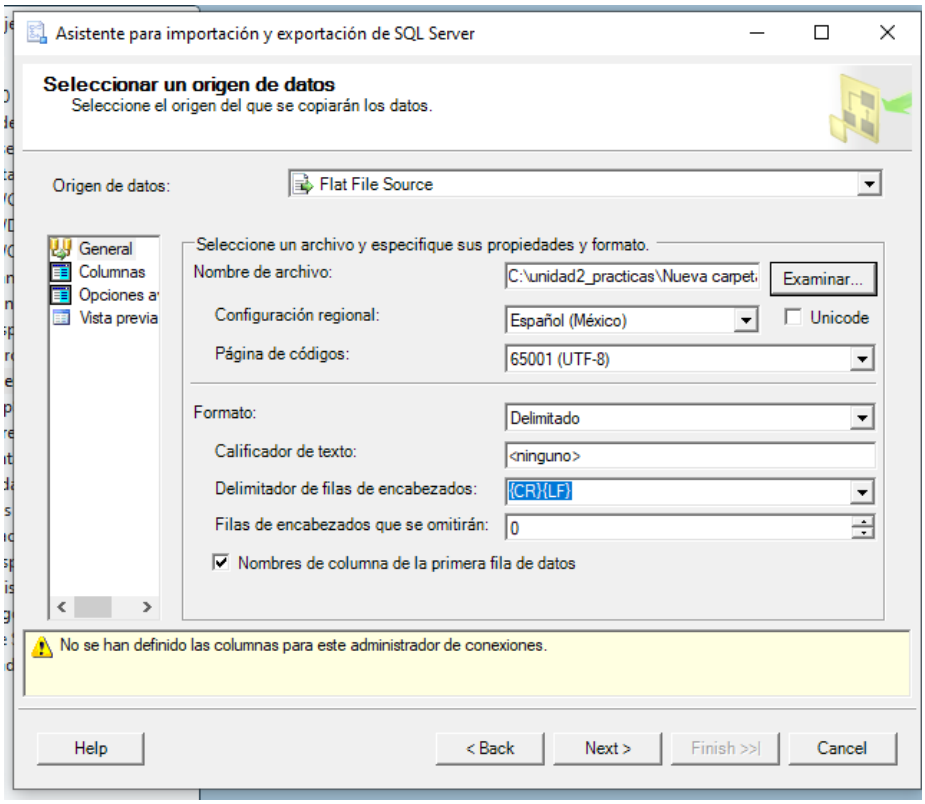


Figura 45. Importación.

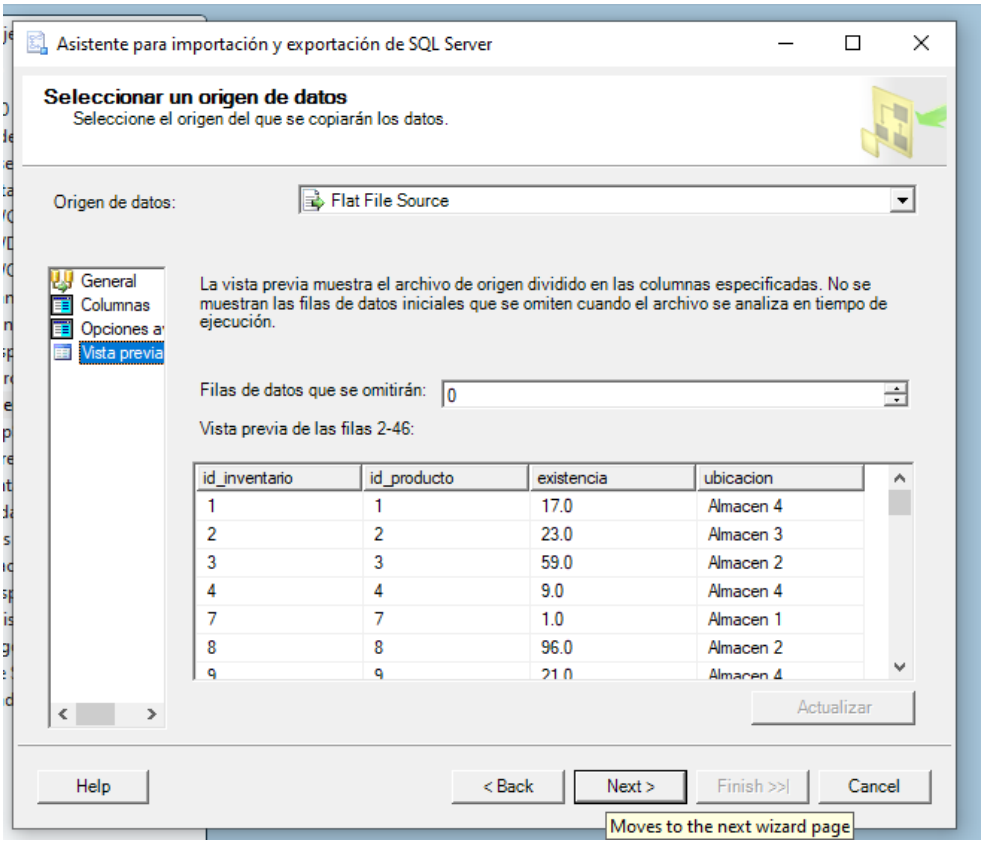


Figura 46. Importación.

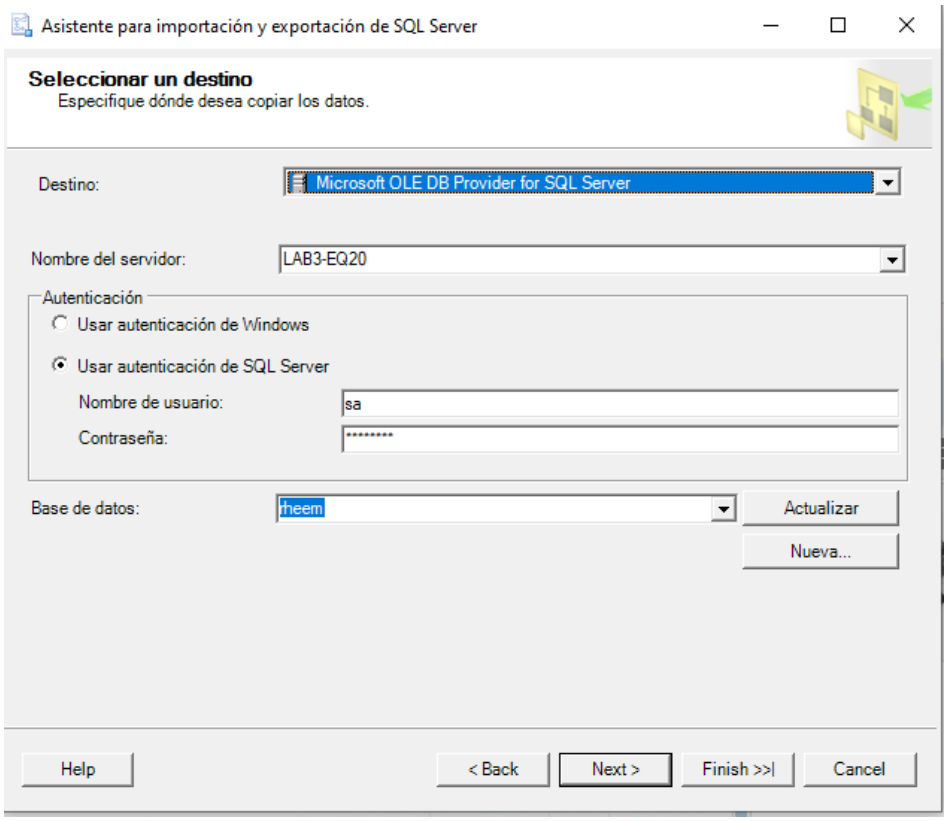


Figura 47. Importación.

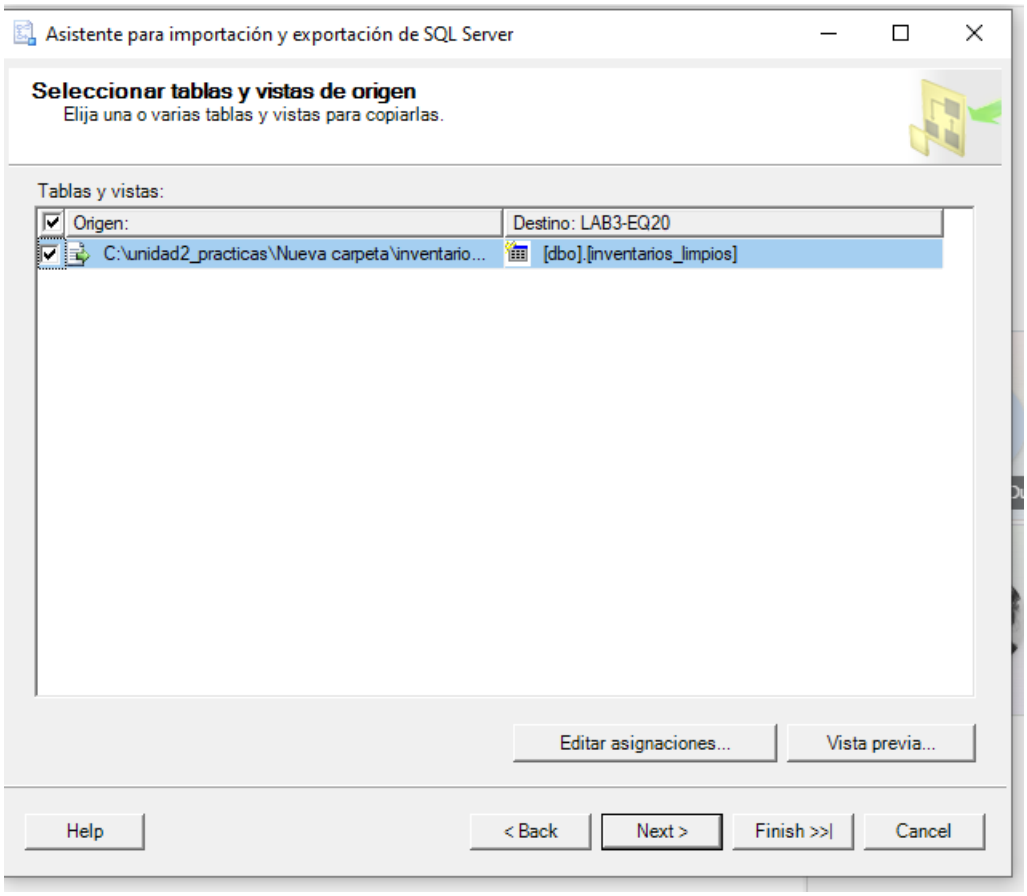


Figura 48. Importación.

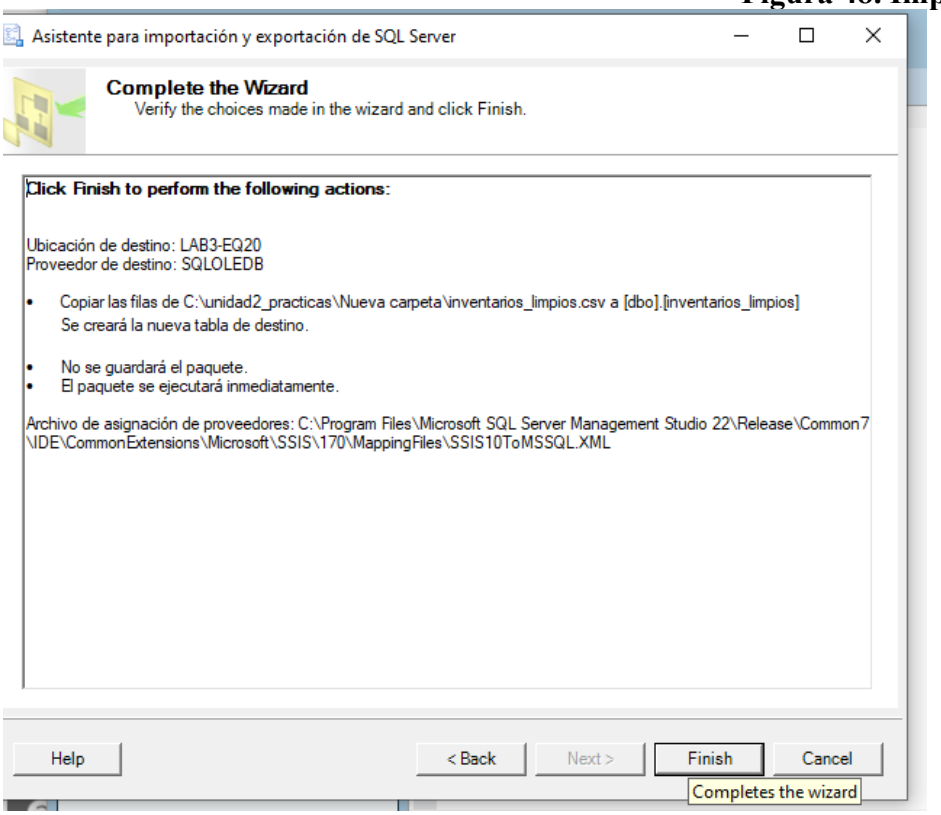


Figura 49. Importación.

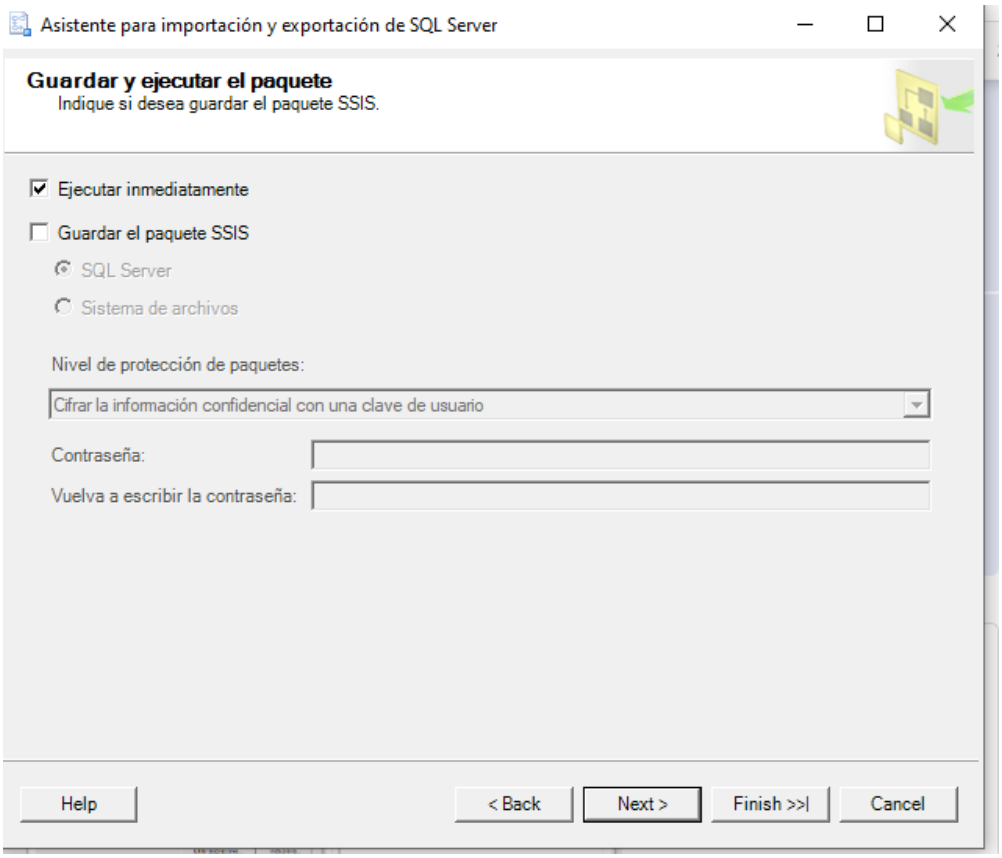


Figura 50. Importación.

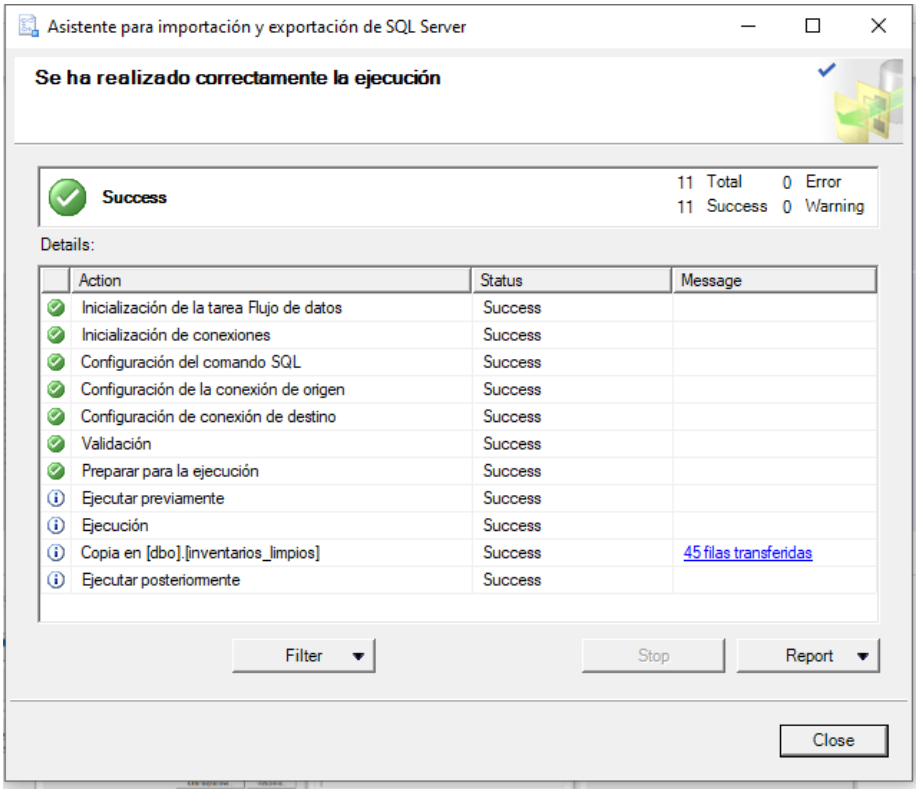
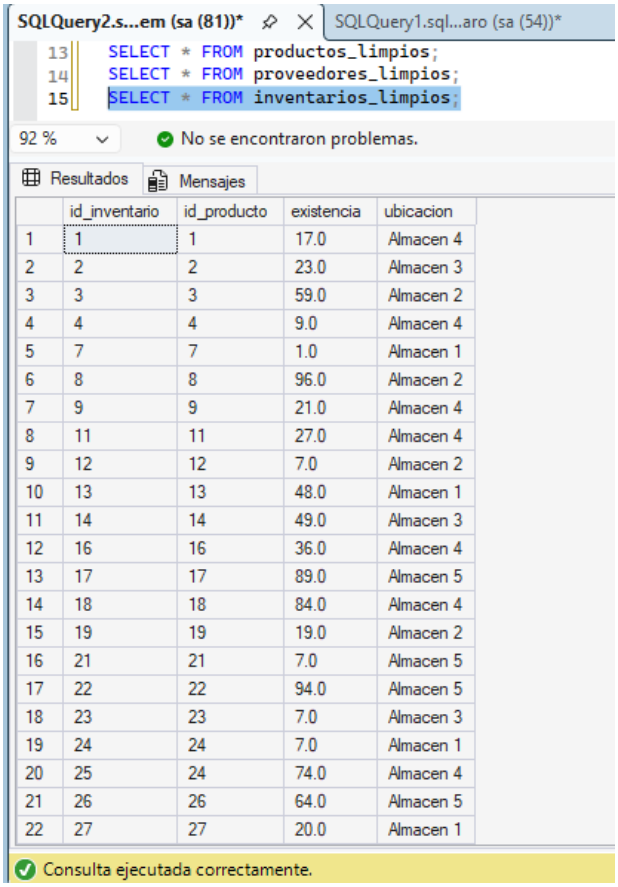


Figura 51. Importación.



	id_inventario	id_producto	existencia	ubicacion
1	1	1	17.0	Almacen 4
2	2	2	23.0	Almacen 3
3	3	3	59.0	Almacen 2
4	4	4	9.0	Almacen 4
5	7	7	1.0	Almacen 1
6	8	8	96.0	Almacen 2
7	9	9	21.0	Almacen 4
8	11	11	27.0	Almacen 4
9	12	12	7.0	Almacen 2
10	13	13	48.0	Almacen 1
11	14	14	49.0	Almacen 3
12	16	16	36.0	Almacen 4
13	17	17	89.0	Almacen 5
14	18	18	84.0	Almacen 4
15	19	19	19.0	Almacen 2
16	21	21	7.0	Almacen 5
17	22	22	94.0	Almacen 5
18	23	23	7.0	Almacen 3
19	24	24	7.0	Almacen 1
20	25	24	74.0	Almacen 4
21	26	26	64.0	Almacen 5
22	27	27	20.0	Almacen 1

Figura 52. Importación.

Productos_limpios.csv

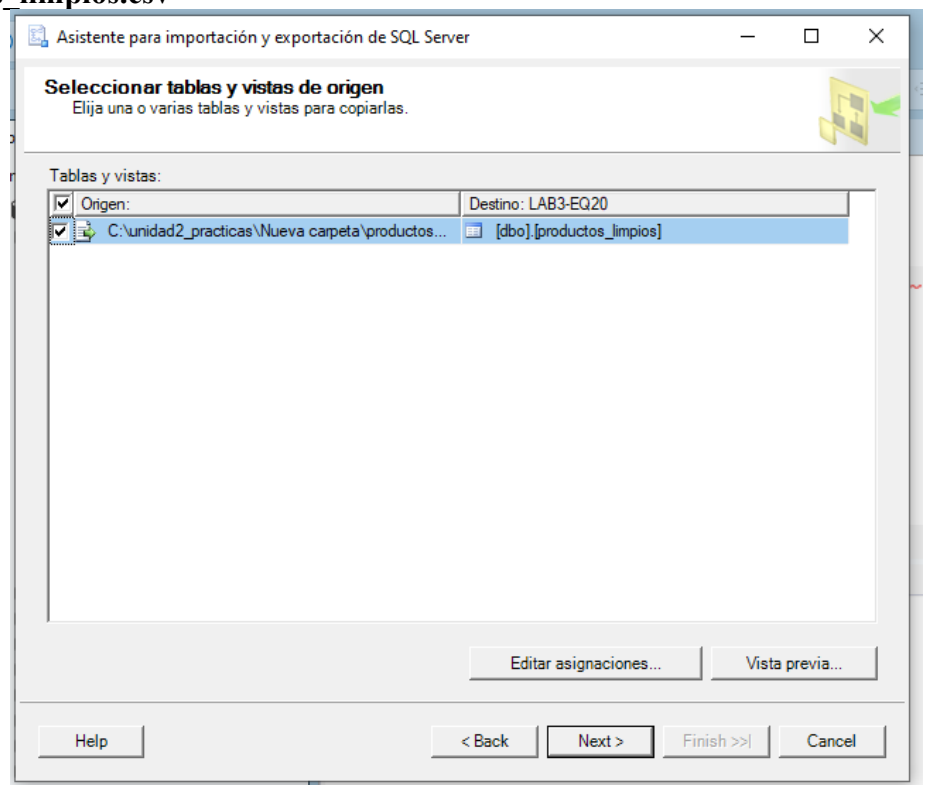


Figura 53. Importación.

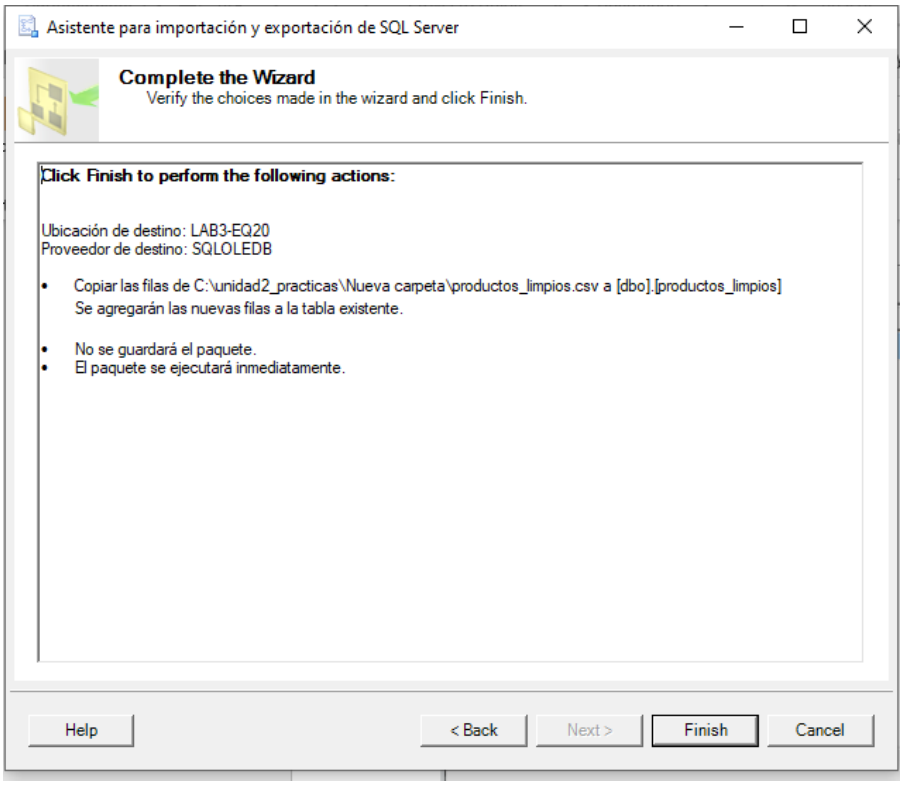


Figura 54. Importación.

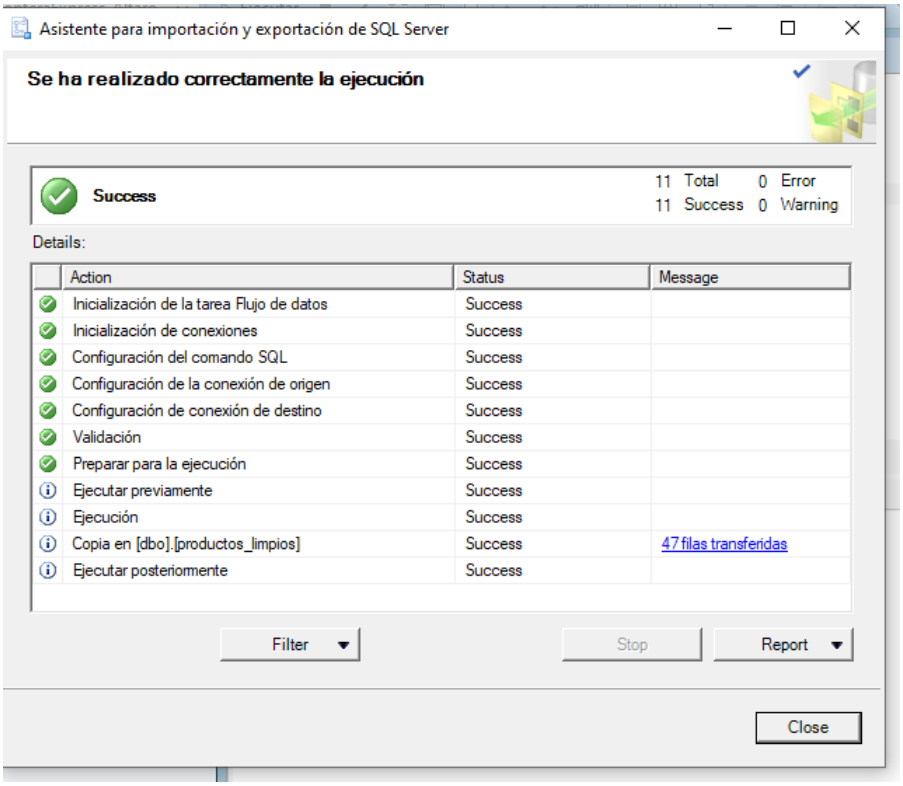


Figura 55. Importación.

SQLQuery2.s...em (sa (81))* | SQLQuery1.sql...aro (sa (54))*

```
13  SELECT * FROM productos_limpios;
14  SELECT * FROM proveedores_limpios;
15  SELECT * FROM inventarios_limpios;
```

92 % — No se encontraron problemas.

Resultados | Mensajes

	id_producto	nombre	categoria	precio	activo
1	1	Calentador de agua Modelo 1	ClimatizaciÃ³n	11252.48	0
2	2	Minisplit Modelo 2	CalefacciÃ³n	11178.83	0
3	3	Condensador Modelo 3	Accesorios	960.85	0
4	4	Minisplit Modelo 4	CalefacciÃ³n	7827.65	0
5	6	Minisplit Modelo 6	Accesorios	9044.35	0
6	7	Termostato Modelo 7	Accesorios	5433.63	0
7	8	Minisplit Modelo 8	Agua caliente	1982.05	1
8	9	Calentador de agua Modelo 9	Agua caliente	12788.67	1
9	11	Calentador de agua Modelo 11	Agua caliente	12526.37	1
10	12	Condensador Modelo 12	CalefacciÃ³n	10716.29	0
11	13	Minisplit Modelo 13	Agua caliente	14785.71	0
12	14	Calentador de agua Modelo 14	Accesorios	4530.62	1
13	15	Sin Nombre	Agua caliente	5651.43	1
14	16	Calentador de agua Modelo 16	CalefacciÃ³n	8245.03	0
15	17	Termostato Modelo 17	Accesorios	6002.1	0
16	18	Bomba de calor Modelo 18	ClimatizaciÃ³n	3821.2	0
17	19	Bomba de calor Modelo 19	Accesorios	4382.24	0
18	21	Filtro de aire Modelo 21	Accesorios	2571.64	0
19	22	Minisplit Modelo 22	Agua caliente	11331.7	1
20	23	Condensador Modelo 23	Accesorios	5748.95	0
21	24	Radiador Modelo 24	Accesorios	1818.19	0
22	25	Calentador de agua Modelo 25		9597.97	1

Figura 56. Importación.

Proveedores_limpios.csv

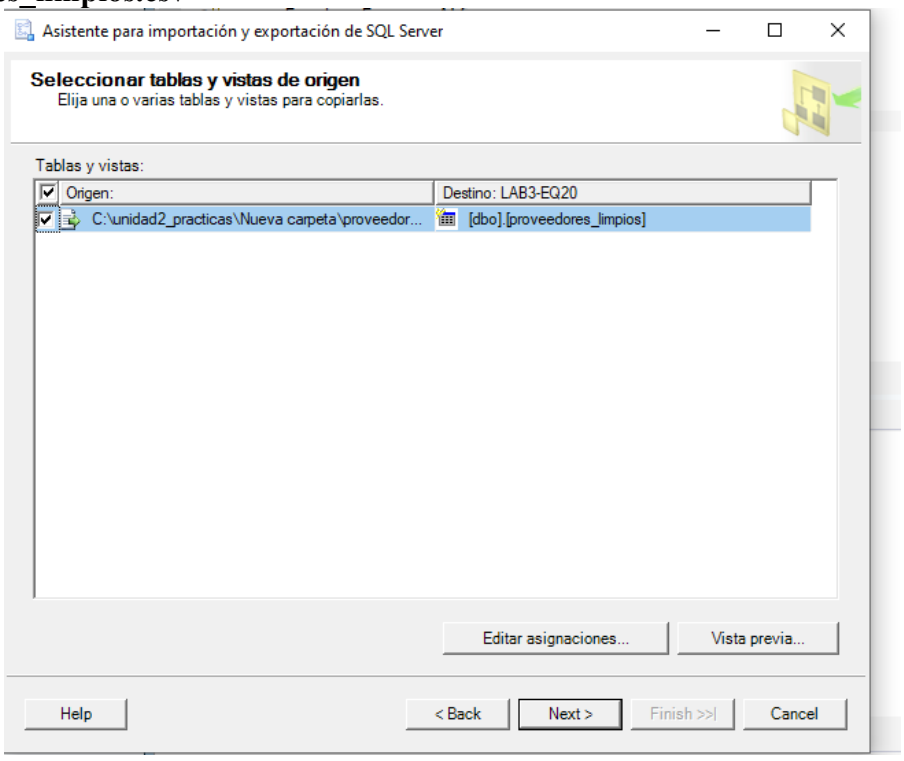


Figura 57. Importación.

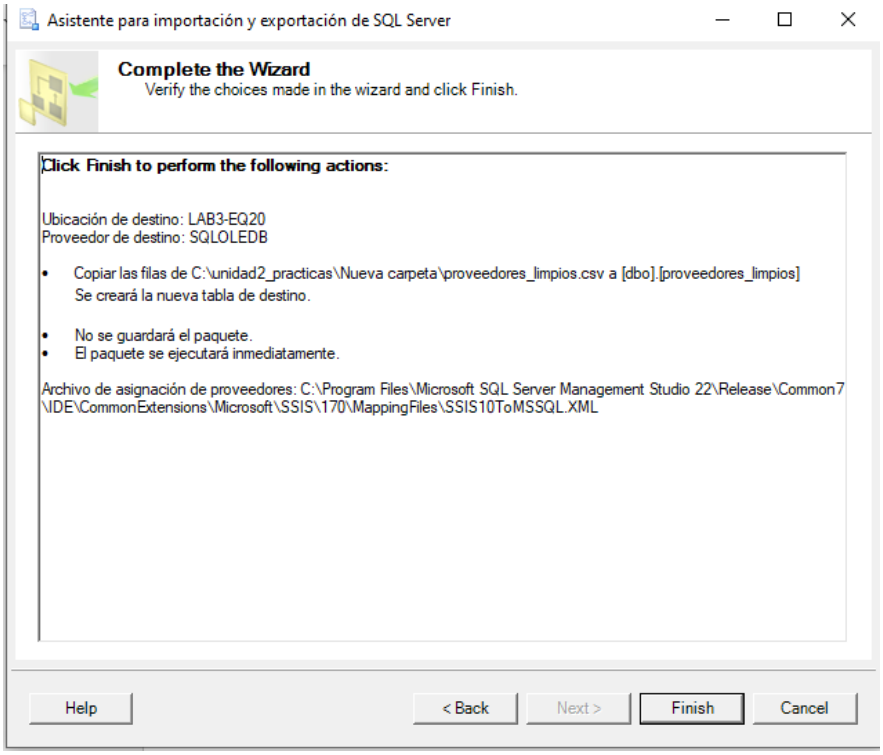


Figura 58. Importación.

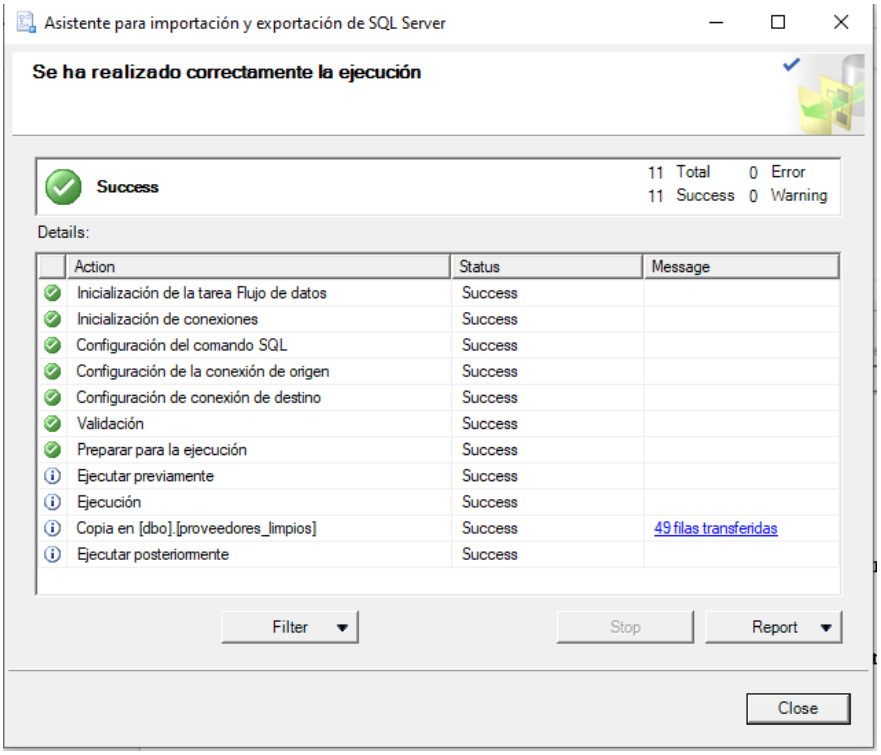
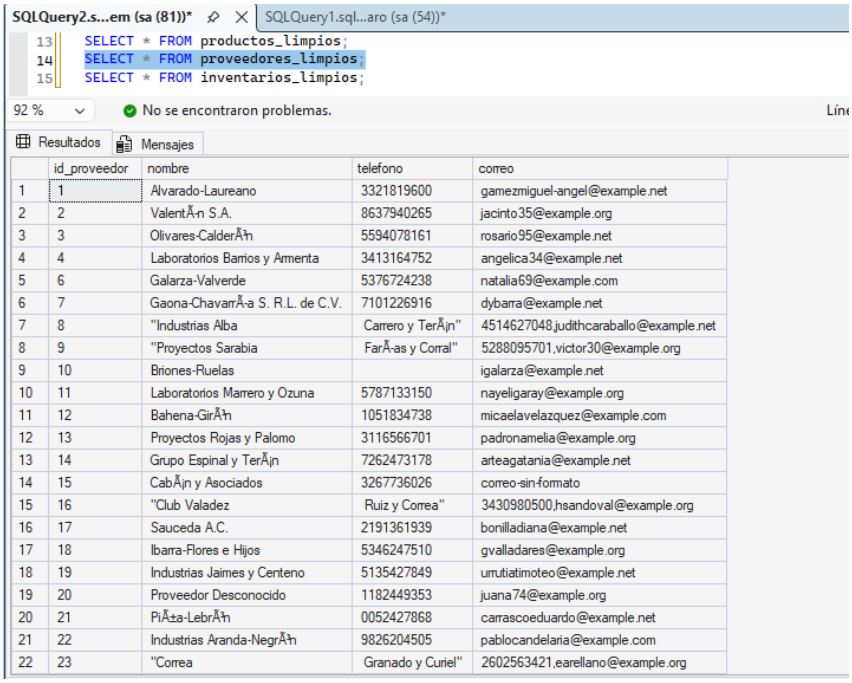


Figura 59. Importación.



SQLQuery2.s...em (sa (81))* | SQLQuery1.sql...aro (sa (54))*

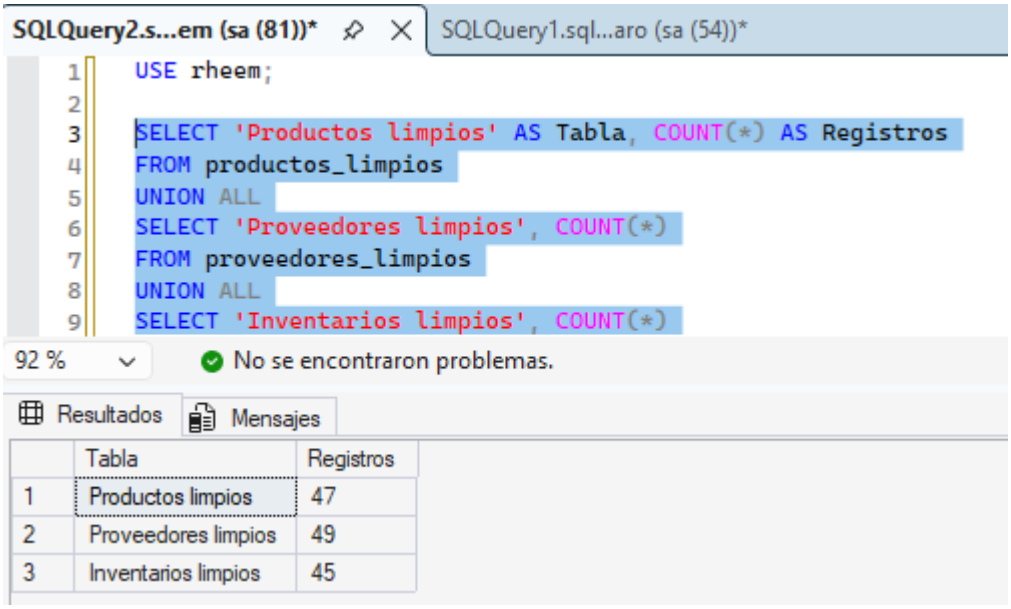
```
SELECT * FROM productos_limpios;
SELECT * FROM proveedores_limpios;
SELECT * FROM inventarios_limpios;
```

92 % — No se encontraron problemas.

Resultados | Mensajes

	id_proveedor	nombre	telefono	correo
1	1	Alvarado-Laureano	3321819600	gamezmiguel-angel@example.net
2	2	ValentÃ-n S.A.	8637940265	jacinto35@example.org
3	3	Olivares-CalderÃ†n	5594078161	rosario95@example.net
4	4	Laboratorios Barrios y Amenta	3413164752	angelica34@example.net
5	6	Galarza-Valverde	5376724238	natalia69@example.com
6	7	Gaona-ChavarrÃ-a S. R.L. de C.V.	7101226916	dybarra@example.net
7	8	"Industrias Alba	Carrero y TerÃjn"	4514627048,judithcaraballo@example.net
8	9	"Proyectos Sarabia	FarÃ-as y Corral"	5288095701,victor30@example.org
9	10	Briones-Ruelas		igalarza@example.net
10	11	Laboratorios Marrero y Ozuna	5787133150	nayeligaray@example.org
11	12	Bahena-GirÃ†n	1051834738	micaelavelazquez@example.com
12	13	Proyectos Rojas y Palomo	3116566701	padronamelia@example.org
13	14	Grupo Espinal y TerÃjn	7262473178	arteagatania@example.net
14	15	CabÃjn y Asociados	3267736026	correo-sin-formato
15	16	"Club Valadez	Ruiz y Correa"	3430980500,hsandoval@example.org
16	17	Sauceda A.C.	2191361939	bonilladiana@example.net
17	18	Ibarra-Flores e Hijos	5346247510	gvalladares@example.org
18	19	Industrias Jaimes y Centeno	5135427849	urrutiatimoteo@example.net
19	20	Proveedor Desconocido	1182449353	juana74@example.org
20	21	PiÃ±a-LebrÃ†n	0052427868	carrascoeduardo@example.net
21	22	Industrias Aranda-NegrÃ†n	9826204505	pablocandelaria@example.com
22	23	"Correa	Granado y Curiel"	2602563421,earellano@example.org

Figura 60. Importación.



SQLQuery2.s...em (sa (81))* | SQLQuery1.sql...aro (sa (54))*

```
USE rheem;

SELECT 'Productos limpios' AS Tabla, COUNT(*) AS Registros
FROM productos_limpios
UNION ALL
SELECT 'Proveedores limpios', COUNT(*)
FROM proveedores_limpios
UNION ALL
SELECT 'Inventarios limpios', COUNT(*)
```

92 % — No se encontraron problemas.

Resultados | Mensajes

	Tabla	Registros
1	Productos limpios	47
2	Proveedores limpios	49
3	Inventarios limpios	45

Figura 61. Importación.

Explicación del procedimiento:
Importación de datos a SQL Server

Una vez finalizado el proceso de limpieza de los datos mediante Python y Pandas, se realizó la importación de los archivos CSV depurados a SQL Server Management Studio (SSMS), con el propósito de almacenar la información y comprobar que los registros pudieran consultarse correctamente.

Procedimiento realizado:

1. Se abrió SQL Server Management Studio y se estableció una conexión con el servidor LAB3-EQ20 mediante autenticación de Windows.
2. Se seleccionó la base de datos rheem y se hizo clic derecho sobre ella para acceder a la opción **Tareas → Importar datos**.
3. En el asistente de importación se seleccionó la opción **Flat File Source**, que permite importar archivos CSV. Posteriormente, se buscó la ubicación de los archivos generados durante la limpieza de datos.
4. Se configuró el destino de SQL Server utilizando el proveedor Microsoft OLE DB Provider for SQL Server y se seleccionó la base de datos rheem.
5. Se realizó la importación de los tres archivos:
    - productos_limpios.csv
    - proveedores_limpios.csv
    - inventarios_limpios.csv
6. Al finalizar el asistente, se generaron las tablas productos_limpios, proveedores_limpios e inventarios_limpios dentro de la base de datos.

Verificación de los datos:

Para comprobar que la importación se realizó correctamente, se ejecutaron consultas SQL mediante la instrucción SELECT * FROM, con las cuales se visualizaron los registros almacenados en cada tabla. También se utilizó la función COUNT(*) para conocer la cantidad de registros importados.

Finalmente, se comprobó que la información de los tres archivos CSV estuviera disponible para su consulta dentro de SQL Server, completando así el proceso de importación de los datos depurados.

2.9 Ejecuta 3 consultas aplicando filtros (where), ordenamiento y columna calculada (SUM, AVG, MIN, MAX, etc.)

Ejecución de consulta 1: Métricas de precios por categoría con IVA y ordenamiento en SQL Server

```sql
-- 1. CONSULTA 1:
/* ============================================================
SELECT
    categoria,
    COUNT(*) AS total_productos,
    AVG(precio) AS precio_promedio,
    AVG(precio * 1.16) AS promedio_con_iva,
    MAX(precio) AS precio_maximo
FROM productos
WHERE precio > 0
GROUP BY categoria
ORDER BY precio_promedio DESC;
```

0 % No se encontraron problemas.

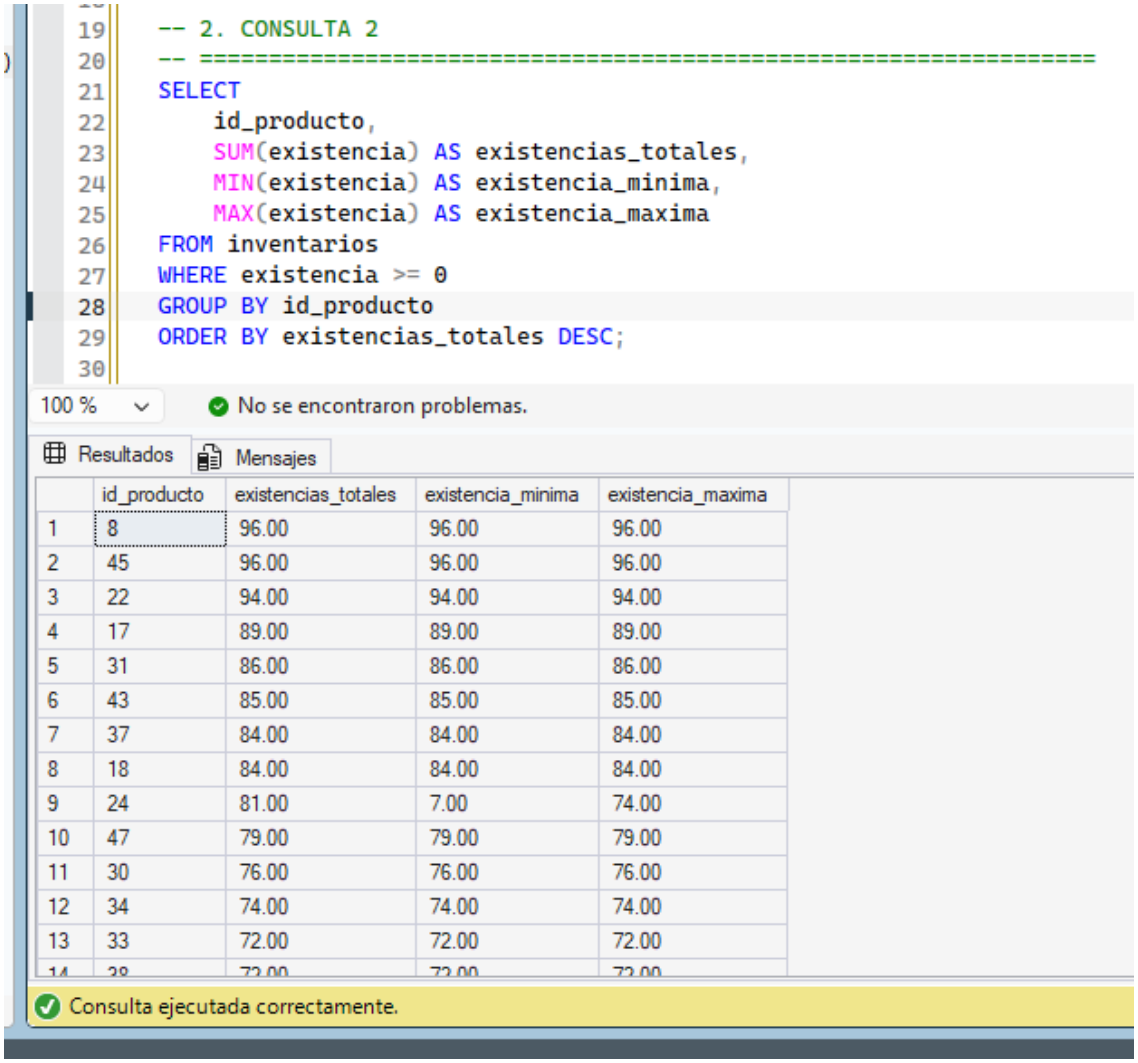
Resultados Mensajes

categoria	total_productos	precio_promedio	promedio_con_iva	precio_maximo
Agua caliente	11	9772.06633966619	11335.5969540128	14785.7099609375
NULL	1	9597.9697265625	11133.6448828125	9597.9697265625
Climatización	10	9144.644921875	10607.788109375	13971.9296875
Accesorios	16	7387.6424331665	8569.66522247314	14929.669921875
Calefacción	9	6676.98117404514	7745.29816189236	11178.830078125

Figura 62. Consultas.

Explicación del impacto: Si existieran datos incorrectos o no depurados, como precios negativos, nulos o valores no numéricos, las funciones AVG(precio) y AVG(precio * 1.16) podrían generar resultados poco confiables. Esto afectaría el análisis de precios por categoría y provocaría errores al calcular el precio promedio de los productos con IVA incluido.

Ejecución de consulta 2: Resumen de existencias totales e inventario mínimo/máximo



```sql
-- 2. CONSULTA 2
-- ================================================================
SELECT
    id_producto,
    SUM(existencia) AS existencias_totales,
    MIN(existencia) AS existencia_minima,
    MAX(existencia) AS existencia_maxima
FROM inventarios
WHERE existencia >= 0
GROUP BY id_producto
ORDER BY existencias_totales DESC;
```

100 % — No se encontraron problemas.

Resultados | Mensajes

	id_producto	existencias_totales	existencia_minima	existencia_maxima
1	8	96.00	96.00	96.00
2	45	96.00	96.00	96.00
3	22	94.00	94.00	94.00
4	17	89.00	89.00	89.00
5	31	86.00	86.00	86.00
6	43	85.00	85.00	85.00
7	37	84.00	84.00	84.00
8	18	84.00	84.00	84.00
9	24	81.00	7.00	74.00
10	47	79.00	79.00	79.00
11	30	76.00	76.00	76.00
12	34	74.00	74.00	74.00
13	33	72.00	72.00	72.00

Consulta ejecutada correctamente.

Figura 63. Consulta.

Explicación del impacto: Si la tabla inventarios contuviera cantidades negativas, valores no numéricos o registros duplicados, las funciones SUM(existencia), MIN(existencia) y MAX(existencia) podrían generar resultados incorrectos. Esto afectaría el control de las existencias y podría ocasionar compras innecesarias o problemas de abastecimiento. Por esta razón, es importante validar los registros antes de realizar las consultas.

Ejecución de consulta 3: Análisis global de importes y precios del catálogo de productos

```sql
-- ============================================================
-- 3. CONSULTA 3:
-- ============================================================
SELECT
    actividad = 'Catálogo Activo',
    SUM(precio) AS valor_total_catalogo,
    AVG(precio) AS precio_promedio_general,
    MIN(precio) AS precio_minimo,
    MAX(precio) AS precio_maximo
FROM productos
WHERE precio > 0
HAVING SUM(precio) > 0
ORDER BY valor_total_catalogo DESC;
```

100 % No se encontraron problemas.

Resultados Mensajes

	actividad	valor_total_catalogo	precio_promedio_general	precio_minimo	precio_maximo
1	Catálogo Activo	386832.258178711	8230.47357827044	604.27001953125	14929.669921875

Figura 64. Consulta.

Explicación del impacto: Si en la tabla productos existieran precios negativos, valores en cero o datos no numéricos, las funciones SUM(precio), AVG(precio) y MIN(precio) podrían generar resultados incorrectos. Esto afectaría el análisis del valor total de los precios registrados en el catálogo y dificultaría establecer referencias para la administración y comercialización de los productos de RHEEM.

2.8 Creación de una tabla temporal y su importancia en la problemática.

```sql
USE rheem;

-- 2. Creación Tabla Temporal
CREATE TABLE #ResumenStockProductos (
    id_producto INT PRIMARY KEY,
    total_piezas INT,
    ubicaciones_distintas INT
);

-- 3. Inserción de los datos
INSERT INTO #ResumenStockProductos (id_producto, total_piezas, ubicaciones_distintas)
SELECT id_producto, SUM(existencia) AS total_piezas, COUNT(DISTINCT ubicacion) AS ubicaciones_distintas
FROM inventarios
WHERE existencia >= 0
GROUP BY id_producto;

-- 4. Consulta de verificación
SELECT *
FROM #ResumenStockProductos
ORDER BY total_piezas DESC;
```

No se encontraron problemas. Línea: 6

Resultados Mensajes

id_producto	total_piezas	ubicaciones_distintas
8	96	1
45	96	1
22	94	1
17	89	1
31	86	1
43	85	1

Activar Windov

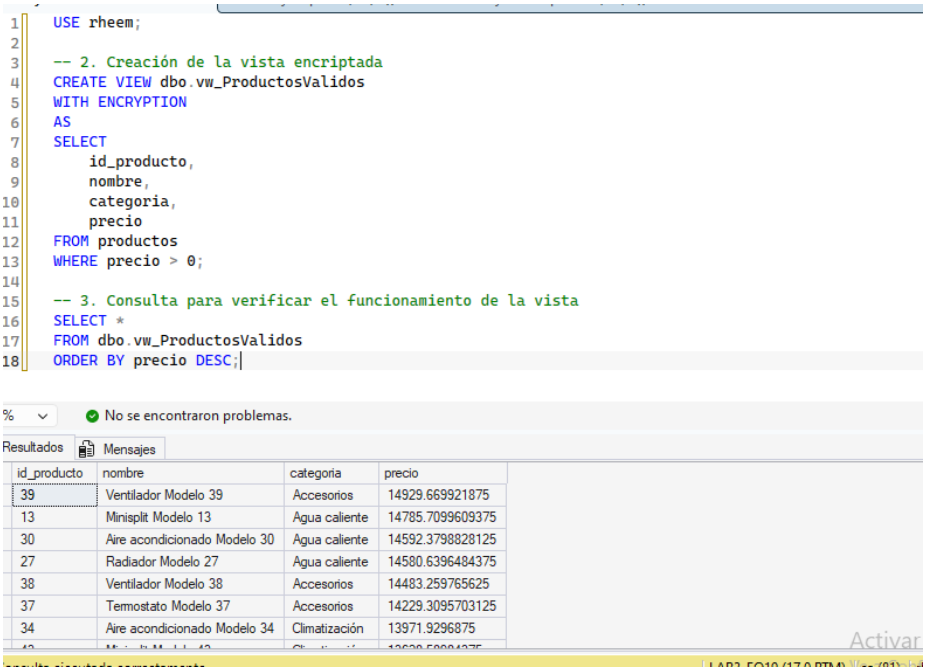
Figura 65. Creación explícita e inserción de datos en la tabla temporal #ResumenStockProductos

Explicación del uso e inserción:

En este paso utilizamos la instrucción CREATE TABLE #ResumenStockProductos para definir primero la estructura de la tabla temporal indicando sus columnas (id_producto, total_piezas, ubicaciones_distintas) y sus tipos de datos. Luego, con el comando INSERT INTO ... SELECT, calculamos las existencias totales y contamos en cuántas ubicaciones se encuentra cada producto para guardar esos datos de manera temporal.

La importancia de utilizar esta tabla temporal en RHEEM es que permite almacenar resultados provisionales del inventario sin modificar las tablas principales. Esto facilita realizar análisis, agrupaciones y revisiones de existencias durante una sesión de trabajo. Al ser una tabla temporal local, SQL Server la administra en tempdb y la elimina automáticamente cuando termina la sesión que la creó. De esta manera, se pueden realizar consultas auxiliares sin conservar información temporal innecesaria.

2.9 Creación de una vista encriptada y su importancia.



```sql
USE rheem;

-- 2. Creación de la vista encriptada
CREATE VIEW dbo.vw_ProductosValidos
WITH ENCRYPTION
AS
SELECT
    id_producto,
    nombre,
    categoria,
    precio
FROM productos
WHERE precio > 0;

-- 3. Consulta para verificar el funcionamiento de la vista
SELECT *
FROM dbo.vw_ProductosValidos
ORDER BY precio DESC;
```

No se encontraron problemas.

Resultados | Mensajes

id_producto	nombre	categoria	precio
39	Ventilador Modelo 39	Accesorios	14929.669921875
13	Minisplit Modelo 13	Agua caliente	14785.7099609375
30	Aire acondicionado Modelo 30	Agua caliente	14592.3798828125
27	Radiador Modelo 27	Agua caliente	14580.6396484375
38	Ventilador Modelo 38	Accesorios	14483.259765625
37	Termostato Modelo 37	Accesorios	14229.3095703125
34	Aire acondicionado Modelo 34	Climatización	13971.9296875

Figura 66. Creación y consulta de la vista encriptada vw_ProductosValidos en SQL Server.

Explicación del impacto de encriptar vistas:

Al utilizar la opción WITH ENCRYPTION al crear una vista, SQL Server oculta su definición para evitar que los usuarios puedan consultar fácilmente el código fuente mediante herramientas habituales como sp_helptext o las vistas del sistema.

En la problemática de RHEEM, esto permite proteger la definición de la consulta utilizada para mostrar los productos válidos y sus precios. Sin embargo, esta opción no sustituye los permisos de seguridad ni impide por sí sola modificar las tablas originales.
Por esta razón, también es necesario administrar correctamente los usuarios y sus privilegios, aplicando el principio del mínimo privilegio para proteger la información de la empresa.

Conclusión

Durante el desarrollo de este proyecto se analizó la importancia de la calidad de los datos en la administración de la información de RHEEM, utilizando los archivos productos.csv, inventarios.csv y proveedores.csv. Mediante Python se identificaron posibles errores, como registros duplicados, campos vacíos, valores numéricos incorrectos y problemas en los datos de contacto.

Posteriormente, se establecieron reglas de negocio para determinar qué registros cumplían con las condiciones necesarias y se aplicó el proceso ETL, que consiste en extraer, transformar y cargar los datos. La limpieza permitió preparar la información para su validación y posterior integración en SQL Server, conservando evidencia de los cambios realizados y de los registros que necesitaron revisión.

La comparación de los datos antes y después de la limpieza permitió evaluar los resultados del proceso y determinar la proporción de registros aptos y no aptos. Asimismo, las consultas SQL facilitaron el análisis de precios y existencias, mientras que la tabla temporal permitió realizar revisiones provisionales y la vista encriptada proporcionó una forma de consultar información mediante una definición protegida frente a la consulta habitual de metadatos.

En conclusión, el uso conjunto de Python, ETL y SQL Server permite mejorar la organización, consistencia y confiabilidad de los datos. Este proceso contribuye a reducir errores que podrían afectar los reportes y la toma de decisiones de la empresa. La calidad de los resultados depende de que las reglas de negocio sean adecuadas, las transformaciones estén justificadas y las validaciones se realicen de manera continua.

Referencias

Company, R. M. (s.f.). *About Rheem.* Obtenido de Rheem.: https://www.rheem.com/about/

Faker. (s.f.). *Faker Documentation*. Obtenido de Faker Documentation: https://faker.readthedocs.io/en/stable/

Foundation, P. S. (s.f.). *Python Documentation*. Obtenido de Python: https://docs.python.org/3/

Jonker, J. A. (2025). *Data Quality Dimensions*. Obtenido de IBM Think: https://www.ibm.com/mx-es/think/topics/data-quality-dimensions

Microsoft. (s.f.). *Apply transaction log backups (SQL Server)*. Obtenido de Microsoft Learn: https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/apply-transaction-log-backups-sql-server

Microsoft. (s.f.). *Create a full database backup (SQL Server)*. Obtenido de Microsoft Learn: https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/full-database-backups-sql-server

Microsoft. (s.f.). *CREATE VIEW (Transact-SQL)*. Obtenido de Microsoft Learn: https://learn.microsoft.com/es-es/sql/t-sql/statements/create-view-transact-sql?view=sql-server-ver17

Microsoft. (s.f.). *Tables*. Obtenido de Microsoft Learn: https://learn.microsoft.com/es-es/sql/relational-databases/tables/tables?view=sql-server-ver17

Microsoft. (s.f.). *Transaction log backups (SQL Server)*. Obtenido de Microsoft Learn: https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/transaction-log-backups-sql-server

Team, p. D. (s.f.). *IO tools (text, CSV, HDF5, ...)*. Obtenido de pandas: https://pandas.pydata.org/docs/user_guide/io.html